EASTERN INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER NETWORKS AND DATA COMMUNICATIONS

PROJECT CSN 304

IMPLEMENTATION OF DATABASE ACCESS CONTROL AND SQL INJECTION PREVENTION IN A MINI ONLINE BANKING SYSTEM

Students

NGO XUAN VINH - 2031220009

NGUYEN VU PHUONG TRANG - 2131220008

TRUONG THI VAN - 2331220034

Supervisor

MSC. NGUYEN NGOC THANH

Abstract

Database security is a critical requirement for modern web applications, especially in financial systems that handle sensitive customer information and transactions. Common threats such as SQL Injection attacks and improper access control can lead to unauthorized access, data leakage, and system compromise.

This project presents the design and implementation of a secure mini online banking system developed for educational and security demonstration purposes. The system focuses on implementing essential security mechanisms, including Role-Based Access Control (RBAC), SQL Injection prevention using Prepared Statements and Sequelize ORM, Least-Privilege database access management, BCrypt password hashing, JSON Web Token (JWT) authentication, CSRF protection using the Double-Submit Cookie pattern, brute-force attack mitigation through account lockout and rate limiting, and comprehensive audit logging.

Security testing was conducted through simulated attack scenarios and authorization validation. The results demonstrate that the implemented security controls effectively mitigate common database-related threats while maintaining system functionality. This project provides practical experience in secure database application development and highlights the importance of integrating multiple security mechanisms to protect web applications.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, MSc. Nguyen Ngoc Thanh, for his valuable guidance, constructive feedback, and continuous support throughout the development of this project. His knowledge and encouragement greatly contributed to the successful completion of this work.

We also would like to thank the lecturers of the School of Computing and Information Technology, Eastern International University, for providing us with the knowledge and practical skills necessary to conduct this project.

In addition, we appreciate the cooperation and dedication of all team members, whose efforts and commitment played an important role in completing the project on time.

Contents

List of Figures

List of Tables

Chapter 1. Introduction

1.1 Problem Statement

Modern web applications heavily depend on database systems to store and process sensitive information. In online banking environments, databases contain highly confidential data such as customer credentials, account balances, transaction records, and personal information. Consequently, database security becomes a critical requirement [1].

However, many database-driven web applications remain vulnerable due to weak access control mechanisms and insecure database query handling. One of the most dangerous threats is SQL Injection (SQLi), which is ranked under OWASP Top 10 A03:2021 - Injection [2]. SQL Injection occurs when an application improperly incorporates user input into SQL statements, allowing attackers to manipulate database queries.

Successful SQL Injection attacks may result in:

- Authentication bypass.
- Unauthorized disclosure of confidential information.
- Modification or deletion of database records.
- Escalation of privileges.
- Complete compromise of the underlying database system.

Besides injection attacks, improper access control is another major security concern. If users are granted excessive privileges, attackers who compromise an account may gain access to sensitive resources beyond their authorization level. Therefore, implementing proper Role-Based Access Control (RBAC) and Least-Privilege principles is essential for protecting financial systems.

This project addresses these challenges by implementing a secure mini online banking system that demonstrates practical techniques for preventing SQL Injection attacks and enforcing secure database access control mechanisms.

1.2 Project Objectives

- Design and implement a secure mini banking system for security demonstration purposes.
- Implement Role-Based Access Control (RBAC) to restrict user permissions based on assigned roles.
- Prevent SQL Injection attacks using Prepared Statements and Sequelize ORM secure query handling.
- Apply the Principle of Least Privilege in MySQL database access management.
- Secure user credentials using BCrypt password hashing.
- Implement brute-force protection through Account Lockout and Rate Limiting mechanisms.
- Protect against Cross-Site Request Forgery (CSRF) attacks using the Double-Submit

Cookie pattern.

- Maintain a comprehensive Audit Logging system for security monitoring and incident investigation.
- Demonstrate the effectiveness of implemented security controls through attack simulations and validation testing.

1.3 Project Scope

This project develops a lightweight online banking application that serves as a security demonstration platform rather than a production-ready banking system.

The system includes only basic banking functionalities such as:

- User authentication.
- Viewing account information.
- Simple money transfer simulation.
- Viewing transaction history.

The primary focus of the project is on security mechanisms rather than business functionality. The project demonstrates practical implementations of:

- Secure authentication and authorization.
- SQL Injection prevention.
- Role-Based Access Control.
- Database privilege management.
- Password protection.
- CSRF defense.
- Brute-force attack mitigation.
- Security auditing and logging.

Advanced banking functions such as payment gateways, multi-factor authentication, fraud detection, and distributed transaction processing are outside the scope of this project.

1.4 Report Structure

Chapter 1 - Introduction: Presents the project background, problem statement, objectives, scope, and report organization.

Chapter 2 - Theoretical Background: Reviews the fundamental security concepts and technologies used in the project.

Chapter 3 - System Design and Architecture: Describes the system architecture, database design, and security framework.

Chapter 4 - Implementation: Explains the implementation details of authentication, authorization, database security, and attack prevention mechanisms.

Chapter 5 - Testing and Evaluation: Demonstrates attack simulations, security testing

procedures, and evaluation results.

Chapter 6 - Conclusion and Future Work: Summarizes project achievements, limitations, and future improvement directions.

Chapter 2. Theoretical Background

2.1 SQL Injection

2.1.1 Definition and Classification

SQL Injection (SQLi) is a code injection attack in which malicious SQL statements are inserted into application inputs and executed by the database server [3].

The attack exploits improper input handling and insecure query construction. SQL Injection can be classified into several categories:

a. Classic (In-Band) SQL Injection

The attacker uses the same communication channel to send malicious payloads and receive responses.

Example:

```
SELECT * FROM users
WHERE username='admin'
AND password='' OR '1'='1';
```

b. UNION-Based SQL Injection

The attacker uses the SQL UNION operator to combine results from multiple queries and retrieve unauthorized data.

Example:

```
' UNION SELECT username,password FROM users--
```

c. Error-Based SQL Injection

The attacker intentionally triggers database errors to reveal information about database structure, tables, or query syntax.

d. Blind SQL Injection

The application does not return database errors or query results directly.

Boolean-Based Blind SQL Injection

The attacker infers information from application behavior based on true or false conditions.

Example:

```
' AND 1=1--
' AND 1=2--
```

Time-Based Blind SQL Injection

The attacker introduces delays and observes response times.

Example:

```
' AND SLEEP(5)--
```

2.1.2 Real-world Impact

SQL Injection remains one of the most damaging web application vulnerabilities.

Common consequences include:

- **Authentication Bypass:** Attackers can log in without valid credentials.
- **Data Extraction:** Sensitive information such as usernames, passwords, financial records, and personal data can be disclosed.
- **Privilege Escalation:** Attackers may gain administrator-level privileges.
- **Data Manipulation:** Records can be modified, inserted, or deleted. Notable examples include:
  - Heartland Payment Systems breach (2008).
  - Sony Pictures breach (2011).
  - Numerous financial institution attacks involving unauthorized access to customer databases.

These incidents demonstrate the severe financial and reputational consequences of insecure database query handling.

2.1.3 Prevention Techniques

Several techniques can effectively prevent SQL Injection attacks [4].

- **Prepared Statements:** Separate SQL commands from user-supplied data, preventing malicious input from altering query structure.
- **ORM Frameworks:** Object Relational Mapping frameworks such as Sequelize automatically parameterize database queries and reduce the risk of injection vulnerabilities.
- **Input Validation:** Applications should validate input format, length, and allowable characters before processing.
- **Web Application Firewall (WAF):** A WAF can detect and block suspicious SQL Injection payloads before they reach the application.

2.2 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is an authorization model that assigns permissions to roles rather than directly to users [5].

The RBAC model consists of:

- Users
- Roles
- Permissions

A user is assigned one or more roles, and each role possesses specific permissions.

In this project, three roles are implemented:

- **Customer:** View own account information, perform money transfer operations, and view personal transaction history.
- **Auditor:** Read-only access to system information, audit logs, and transaction records.
- **Administrator:** Manage users and permissions, monitor system activity, and access administrative functions.

RBAC simplifies permission management and supports security principles such as least privilege and separation of duties.

2.3 Principle of Least Privilege

The Principle of Least Privilege (PoLP) states that users, processes, and applications should receive only the minimum permissions required to perform their assigned tasks.

Applying least privilege reduces:

- Attack surface.
- Accidental data exposure.
- Impact of compromised accounts.

In MySQL environments, least privilege can be implemented by creating separate database accounts with limited permissions [6].

Examples:

Customer Database User

```
GRANT SELECT ON banking.accounts TO customer_user;
```

Auditor Database User

```
GRANT SELECT ON banking.* TO auditor_user;
```

Administrator Database User

```
GRANT ALL PRIVILEGES ON banking.* TO admin_user;
```

This approach ensures that users cannot perform unauthorized database operations.

2.4 Password Hashing with BCrypt

Password hashing converts plaintext passwords into irreversible cryptographic representations.

Unlike encryption, hashing is a one-way process.

BCrypt is widely used because it provides:

- **Salt Generation:** A unique random salt is automatically generated for every password.

- **Adaptive Cost Factor:** The computational cost can be increased as hardware becomes more powerful.
- **Resistance to Rainbow Table Attacks:** Unique salts prevent attackers from using precomputed hash databases.

Example:

```
const hash = await bcrypt.hash(password, 12);
```

Compared with MD5 and SHA-1, BCrypt provides significantly stronger protection against password cracking attacks [7].

2.5 Authentication and Session Security

2.5.1 JSON Web Token (JWT)

JSON Web Token (JWT) is a compact authentication token format used for securely transmitting user identity information [8].

A JWT consists of three parts:

- **Header:** Contains token type and signing algorithm.
- **Payload:** Contains claims such as user ID, username, role, and expiration time.
- **Signature:** Used to verify token integrity.

The token includes expiration timestamps to limit session duration and is delivered through secure cookies.

2.5.2 Secure Cookie Flags

Secure cookie attributes help protect authentication tokens [9].

- **HttpOnly:** Prevents JavaScript access to cookies, reducing XSS risks.
- **SameSite=Strict:** Prevents browsers from automatically sending cookies during cross-site requests, mitigating CSRF attacks.
- **Secure:** Ensures cookies are transmitted only over HTTPS connections.

Example:

```
res.cookie("token", jwt, {
    httpOnly: true,
    secure: true,
    sameSite: "strict"
});
```

2.6 CSRF Attack and Prevention

2.6.1 How CSRF Works

Cross-Site Request Forgery (CSRF) occurs when an authenticated user unknowingly submits a malicious request to a trusted application [10].

Attack scenario:

- User logs into the banking system.
- Authentication cookie remains active.
- User visits a malicious website.
- The malicious website automatically submits a forged transfer request.
- Browser sends authentication cookies automatically.
- The banking system processes the unauthorized request.

2.6.2 Double-Submit Cookie Pattern

The Double-Submit Cookie pattern protects against CSRF by requiring a matching token in two locations:

- CSRF cookie stored by the browser.
- CSRF token included in request headers.

Verification process:

Cookie Token == Header Token

In this project, the CSRF token is additionally bound to a browser-specific identifier, making token theft and reuse significantly more difficult.

Only requests with matching tokens are accepted by the server.

2.7 Brute Force Attack and Prevention

2.7.1 Account Lockout Policy

Brute-force attacks attempt to guess user credentials through repeated login attempts.

To mitigate this threat, the system implements an Account Lockout Policy.

Policy:

- Maximum 5 consecutive failed login attempts.
- Account locked for 5 minutes after threshold exceeded.
- Failed attempt counters stored in the database.
- Counter resets after successful authentication.

This mechanism prevents automated password guessing attacks against individual accounts.

2.7.2 Rate Limiting

Rate limiting restricts the number of requests accepted from a specific IP address within a time window.

The project uses Express Rate Limit with the following policy:

- Maximum Requests: 10.
- Time Window: 10 minutes.

Example configuration:

```
const limiter = rateLimit({
  windowMs: 10 * 60 * 1000,
  max: 10
});
```

Rate limiting reduces automated attack effectiveness and protects server resources.

2.8 Audit Logging

Audit logging records security-related events generated by the system.

The primary objectives are:

- Security monitoring.
- Incident detection.
- Forensic investigation.
- Regulatory compliance.

Events recorded in this project include:

- User login attempts.
- Failed authentication events.
- Account lockouts.
- Money transfer transactions.
- Administrative actions.
- Access-denied events.
- Security violations.

Typical audit log fields include:

- Timestamp.
- User ID.
- User role.
- Source IP address.
- Action performed.
- Result status.

Audit logs provide accountability and support post-incident analysis by enabling investigators to reconstruct security events and identify malicious activities.

Chapter 3. System Design

3.1 System Architecture

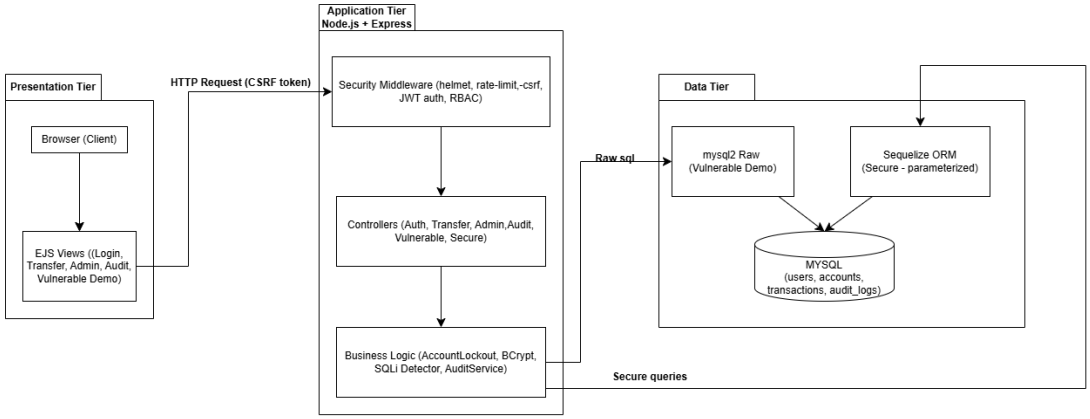


Figure 3.1: Three-Tier System Architecture

The system uses a three-tier architecture: Presentation, Application, and Data tier.

Presentation Tier is the browser side, rendering EJS templates served from the backend. Pages include Login, Transfer, Admin Panel, Audit Log, and the SQL Injection demo. Every form has a hidden CSRF token field that gets submitted along with the request.

Application Tier runs on Node.js + Express, split into three layers:

- **Security Middleware** sits in front of every request - it sets HTTP security headers via Helmet, checks the CSRF token, verifies the JWT from the HttpOnly cookie, enforces role-based access control, and blocks excessive login attempts with rate limiting.
- **Controller Layer** handles the actual route logic: login/logout, fund transfer, user management, audit log retrieval, and the vulnerable/secure demo endpoints.
- **Business Logic** is embedded inside controllers: account lockout after 5 failed attempts, BCrypt password check, SQL injection pattern detection, and writing audit events.

Data Tier has two separate database connections. mysql2 raw is used only by the vulnerable demo endpoints - it builds queries through string concatenation, which makes injection possible. Sequelize ORM handles everything else - it uses parameterized queries so user input is never treated as SQL. Both connect to the same MySQL 8 instance with four tables: users, accounts, transactions, and audit_logs. Four least-privilege MySQL users are configured, each mapped to a role, so even if injection succeeds the damage is limited by what that DB user is actually allowed to do.

3.2 Technology Stack

Table 3.1: Technology Stack

Component	**Technology**	**Purpose**
Backend Framework	Node.js + Express 5	REST API, routing
ORM (secure)	Sequelize 6	Parameterized queries
Raw query (demo)	mysql2	Vulnerable endpoint demo
Database	MySQL 8.x	Data storage
Password hashing	bcryptjs	Credential protection
Authentication	JWT + cookie	Session management
Security headers	helmet	HTTP header hardening
CSRF protection	csrf-csrf	Double-submit cookie
Rate limiting	express-rate-limit	Brute force prevention
Template engine	EJS	Server-side rendering

3.3 Database Design

3.3.1 Entity Relationship Diagram

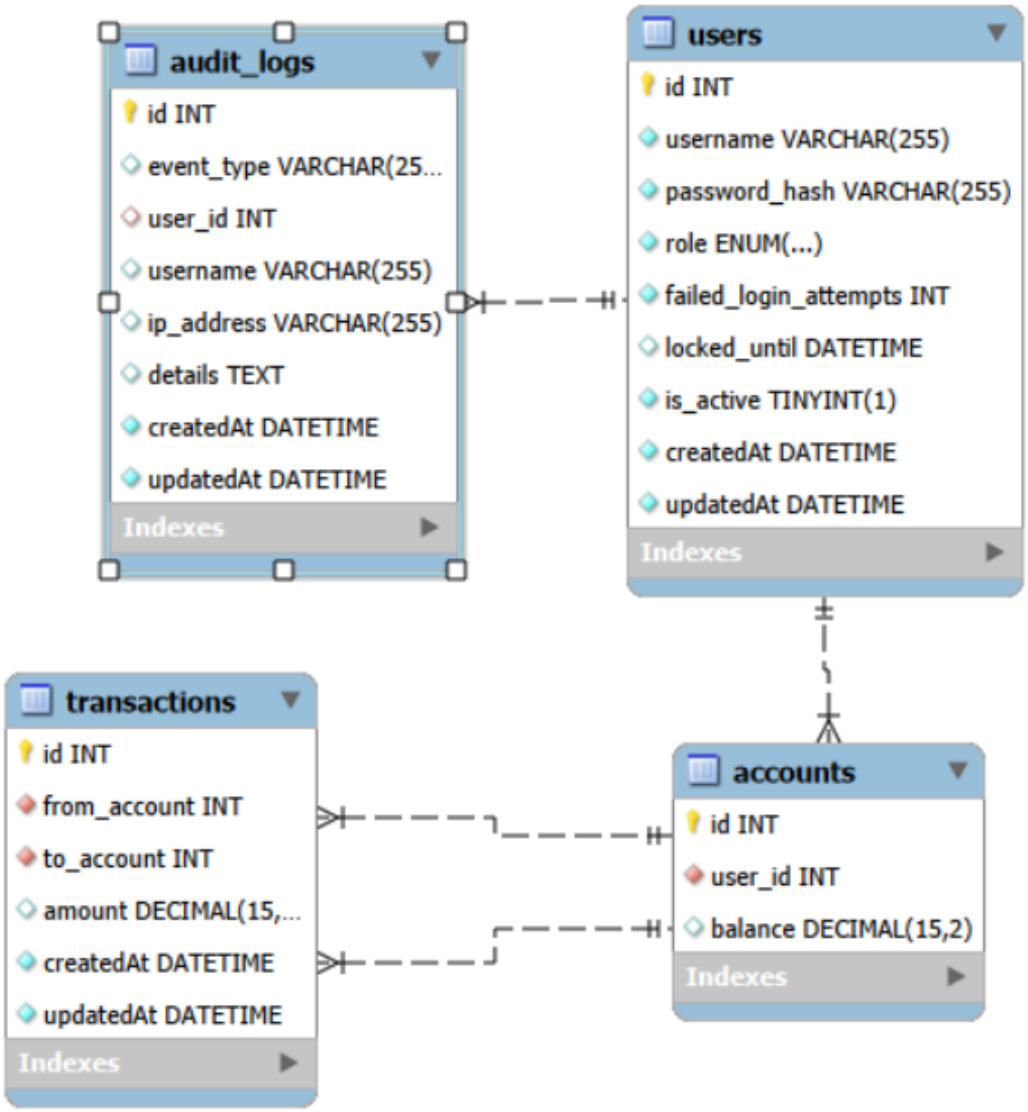


Figure 3.2: Entity Relationship Diagram

The database has four tables. **users** stores account credentials and security-related fields like failed_login_attempts, locked_until, and is_active to support lockout behavior. Each user has one **accounts** record holding their balance, linked via user_id. **transactions** records every fund transfer with from_account and to_account referencing accounts.id, so both sides of a transfer are tracked. **audit_logs** is independent - it logs security events by username and user_id without a hard foreign key, so logs are preserved even if a user gets deleted.

3.3.2 Table Descriptions

Table 3.2: Table: users

Column	Type	Description
id	INT AUTO_INCREMENT	Primary key
username	VARCHAR(255)	Login identifier
password_hash	VARCHAR(255)	BCrypt hash
role	ENUM	CUSTOMER / ADMIN / AUDITOR
failed_login_attempts	INT DEFAULT 0	Brute-force counter
locked_until	DATETIME NULL	Temporary lockout expiry
is_active	BOOLEAN DEFAULT 1	Admin-controlled disable
createdAt	DATETIME	Times when the record was created.
updatedAt	DATETIME	Timestamp of the last update.

Table 3.3: Table: accounts

Column	Type	Description
id	INT AUTO_INCREMENT	Primary key
user_id	INT NOT NULL	Foreign key to Users table
balance	DECIMAL(15,2) DEFAULT '0.00'	Remaining balance of account

Table 3.4: Table: transactions

Column	Type	Description
id	INT AUTO_INCREMENT	Primary key
from_account	INT NOT NULL	Foreign key to Accounts table
to_account	INT NOT NULL	Foreign key to Accounts table
amount	DECIMAL (15,2) DEFAULT NULL	The amount of the transaction
createdAt	DATETIME	Times when the record was created.
updatedAt	DATETIME	Timestamp of the last update.

Table 3.5: Table: audit_logs

Column	Type	Description
id	INT AUTO_INCREMENT	Primary key
event_type	VARCHAR(255) DEFAULT NULL	Event category (LOGIN, LOGOUT...)
user_id	INT DEFAULT NULLL	Foreign key to Users table
username	VARCHAR(255) DEFAULT NULL	Login identifier
ip_address	VARCHAR(255) DEFAULT NULL	IP address of the device
details	TEXT	Detailed description for event
createdAt	DATETIME	Times when the record was created.
updatedAt	DATETIME	Timestamp of the last update.

3.4 RBAC Model Design

Table 3.6: Role-Permission Matrix

Permission	CUSTOMER	ADMIN	AUDITOR
View own account	x	x	x
Transfer funds	x		
View own transactions	x		
Manage all users		x	
View audit logs			x

3.5 Security Architecture Overview

3.5.1 Post/Login Activity Diagram

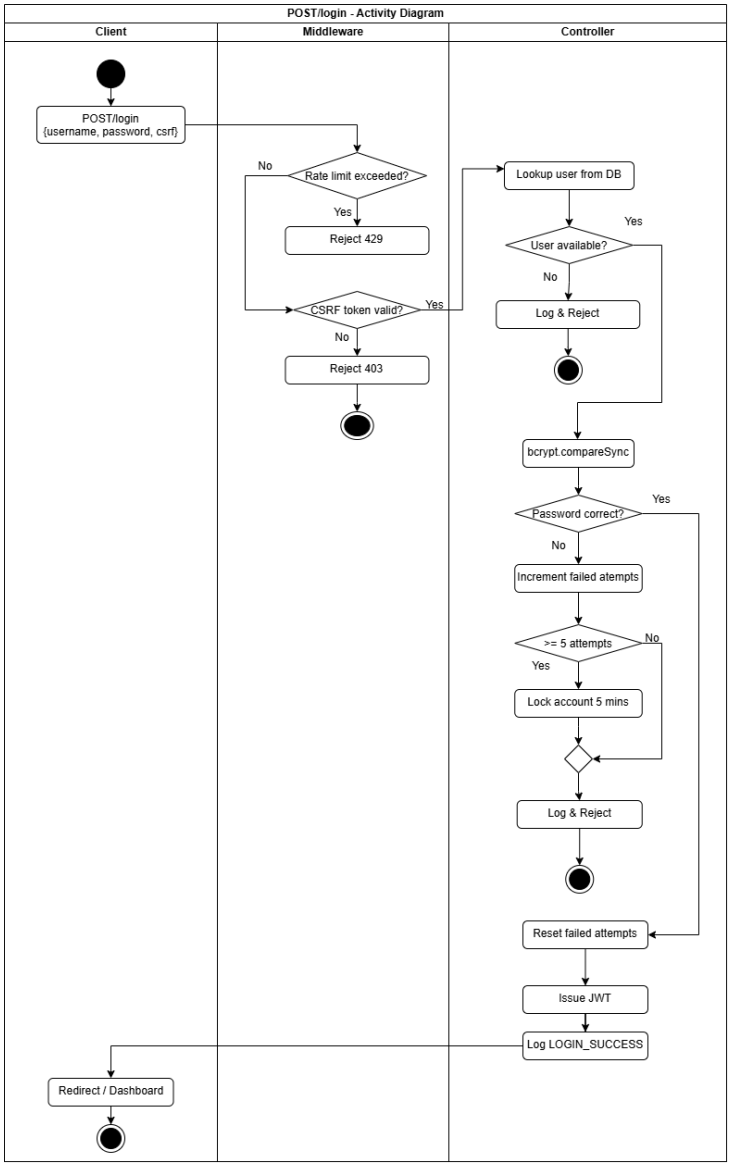


Figure 3.3: Post/login Activity Diagram

The diagram shows the full login flow across three swimlanes: Client, Middleware, and Controller.

When the client sends POST /login, the Middleware checks rate limit first - if the IP has exceeded 10 failed attempts within 10 minutes it returns 429 immediately. If not, it validates the CSRF token; an invalid or missing token gets rejected with 403.

Once both checks pass, the Controller takes over. It looks up the user in the database - if the username does not exist, it logs the event and rejects. If the user exists, it runs bcrypt.compareSync to check the password. A wrong password increments failed_login_attempts in the DB, then checks if the count has hit 5 - if yes the account gets locked for 5 minutes, either way it logs and rejects.

If the password is correct, failed_login_attempts resets to 0, a JWT is issued and stored in an HttpOnly cookie, a LOGIN_SUCCESS event is written to audit_logs, and the client gets redirected to the dashboard.

3.5.2 POST/transfer Activity Diagram

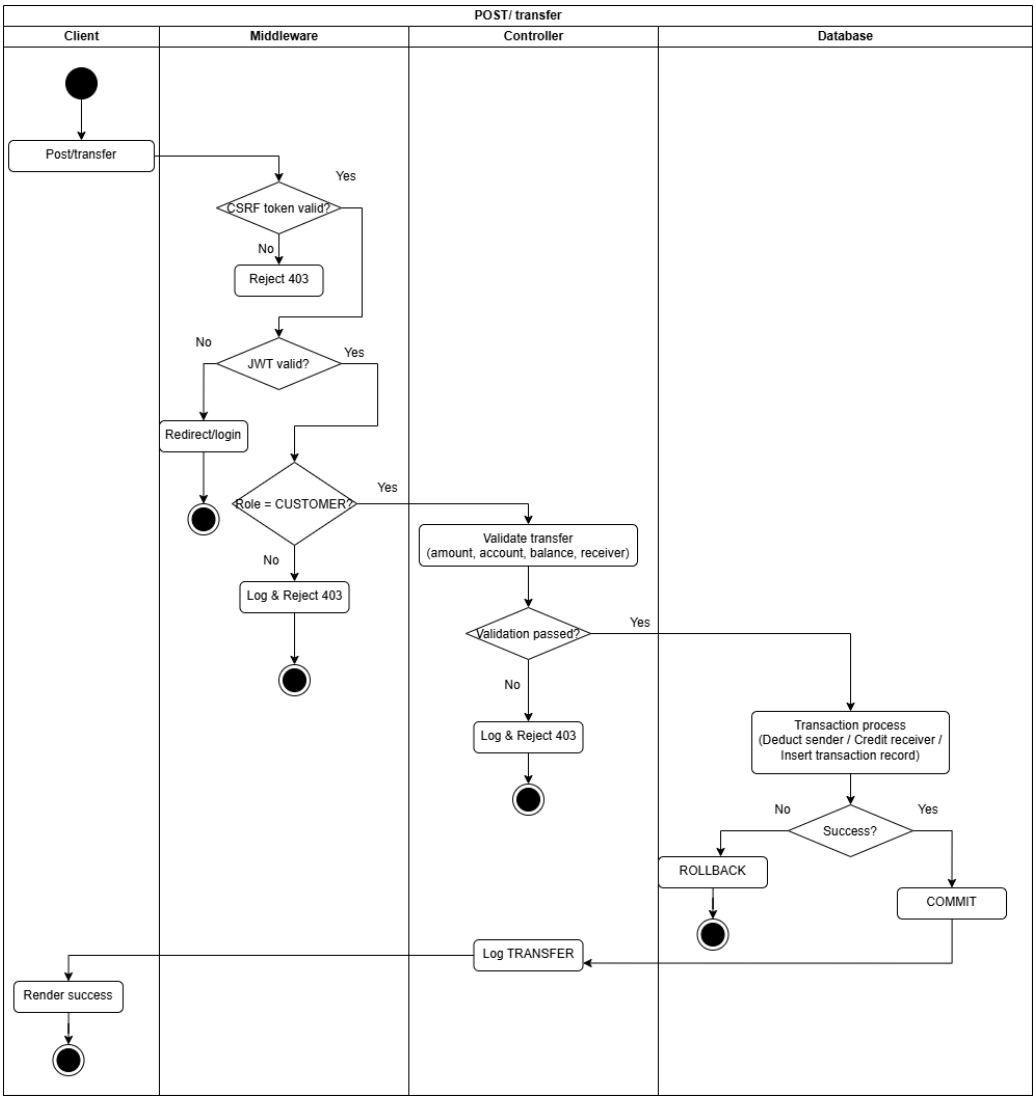


Figure 3.4: Post/transfer Activity Diagram

The diagram covers the transfer flow across four swimlanes: Client, Middleware, Controller, and Database.

The request hits Middleware first. CSRF token is checked - invalid token returns 403 right away. Then JWT is verified - if missing or expired the user gets redirected to login. After that, the role is checked: only CUSTOMER can proceed, anything else gets logged and rejected with 403.

Once all three middleware checks pass, the Controller validates the transfer inputs: amount must be positive and under 100 million, receiver account must exist, sender must not be transferring to themselves, and balance must be sufficient. If any of these fail, it logs and rejects.

If validation passes, the Database swimlane takes over. It opens a transaction with SELECT FOR UPDATE to lock both accounts, deducts from the sender, credits the receiver, and inserts a transaction record. If anything goes wrong at this stage, the whole thing rolls back. On success it commits.

After commit, the Controller logs a TRANSFER event to audit_logs and sends the success response back to the client.

3.5.3 Secure - Vulnerable SQL Injection Activity Diagram

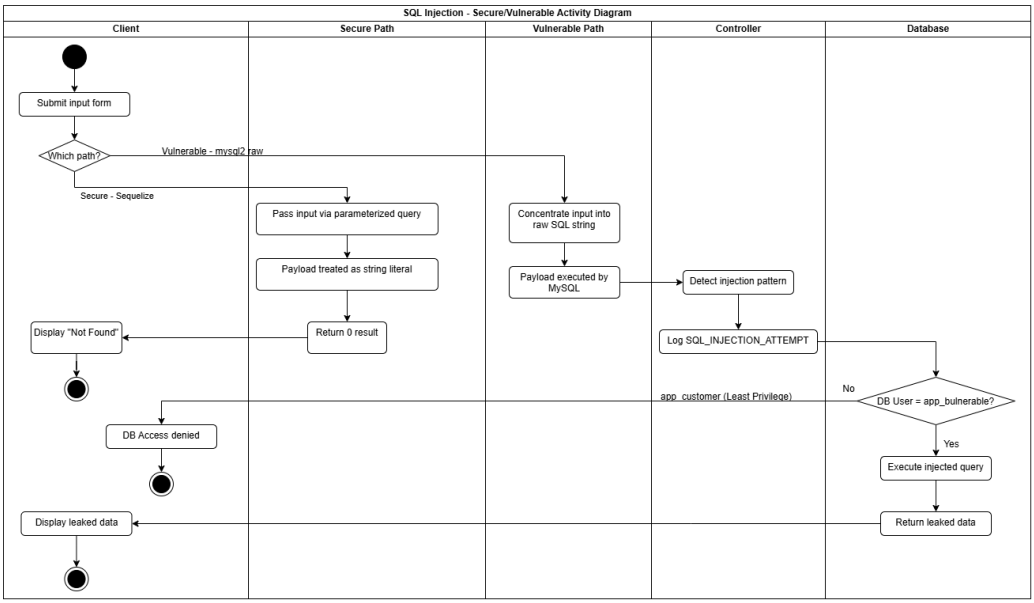


Figure 3.5: Secure - Vulnerable SQL Injection Activity Diagram

The diagram shows what happens when a user submits an input form, depending on which path the request goes through - Secure (Sequelize) or Vulnerable (mysql2 raw).

Secure Path: Input is passed through a parameterized query, so whatever the user types gets treated as a plain string literal. MySQL never sees it as SQL syntax. Result comes back empty, client gets "Not Found."

Vulnerable Path: Input gets concatenated directly into a raw SQL string and sent to MySQL. MySQL executes whatever is in there, including any injected payload. The Controller detects the injection pattern and logs a SQL_INJECTION_ATTEMPT event to audit logs regardless. Then

it checks which DB user is active - if it is app_vulnerable (full privileges), the injected query runs and returns leaked data to the client. If it is app_customer (least privilege), MySQL denies the query at the DB level and the client gets an access denied error instead.

The key point the diagram shows is that two defenses work independently: parameterized queries stop injection at the query level, and least-privilege stops it at the database permission level - even if the query somehow gets through.

3.6 MySQL Least-Privilege Design

Table 3.7: MySQL Database Users and Privileges

MySQL User	**Granted On**	**Privileges**
app_customer	transactions	SELECT, INSERT
app_customer	accounts	SELECT, UPDATE
app_auditor	All tables	SELECT only
app_admin	users	SELECT, INSERT, UPDATE, DELETE
app_admin	accounts	SELECT, INSERT, UPDATE
app_vulnerable	All tables	ALL PRIVILEGES

Chapter 4. Security Implementation

4.1 Authentication Module

4.1.1 JWT Authentication and Secure Cookie Storage - auth.controller.js

- Login flow:

1. Validate user available (existing, `is_active` status, `locked_until` time).
2. Extracts the salt embedded in the stored hash, re-hashes the input using that salt, then compares the result.

```
108
109         const passwordValid = bcrypt.compareSync(password, user.password_hash);
110
```

3. Validate password, if not match then process `failed_login_attempts`, lock account, log event.
4. Else create log success, generate new token, set to cookie.

```
171
172         const token = jwt.sign(
173             {
174                 id: user.id,
175                 username: user.username,
176                 role: user.role,
177             },
178             process.env.JWT_SECRET,
179             {
180                 expiresIn: '1h',
181             }
182         );
183
184         res.cookie('token', token, {
185             httpOnly: true,
186             sameSite: 'strict',
187             secure: process.env.NODE_ENV === 'production',
188             maxAge: 60 * 60 * 1000,
189         });
190
```

- Cookie flags:

+ *httpOnly: true* - Cookie is inaccessible to JavaScript, prevents XSS attacks from stealing the token via `document.cookie`

+ *sameSite: 'strict'* - Cookie is only sent on same-site requests, blocks cross-site request forgery even without an explicit CSRF token
+ *Secure:* Cookie is only transmitted over HTTPS in production - prevents token interception over unencrypted HTTP
+ *maxAge: 3600000* - Token expires after 1 hour, limits the attack window if the token is compromised

4.1.2 Safe Redirect Validation

When a user accesses a protected route while unauthenticated, the application stores the intended destination in the ?redirect= query parameter and redirects back after login. Without validation, an attacker could craft a malicious link such as ?redirect=https://evil.com to redirect victims to a phishing site after login - known as an Open Redirect attack.

getSafeRedirect() validates the redirect value before use: it only accepts paths that start with / and rejects protocol-relative URLs starting with //, which browsers interpret as https://evil.com.

```
9
10  const getSafeRedirect = (req) => {
11      const requestedRedirect = req.query.redirect;
12
13      if (
14          requestedRedirect &&
15          requestedRedirect.startsWith('/') &&
16          !requestedRedirect.startsWith('//')
17      ) {
18          return requestedRedirect;
19      }
20
21      return '/dashboard';
22  };
23
```

4.2 Password Protection - BCrypt Hashing

Passwords in the database should not be stored in plain text. Because once the database is breached, all credentials are immediately exposed. Bcrypt is a one way hashing algorithm - it is irreversible, meaning the original password can not be recovered from the hash. Each hash contains an embedded salt - a random value generated uniquely per user. This means two users with the same password will produce entirely different hashes, rendering rainbow table attack ineffective.

Salt rounds (set to 10 in this project) control the cost factor of the algorithm. BCrypt runs 2^{10} = 1,024 iterations internally before producing a hash. This makes each hashing operation take

roughly 100ms - negligible for a legitimate login, but computationally expensive enough to make brute-force attacks impractical at scale.

- Hashing password of new created user:

```
27      const password_hash = bcrypt.hashSync(u.password, 10);
```

- Get embedded salt, hashing and compare.

```
108
109     const passwordValid = bcrypt.compareSync(password, user.password_hash);
```

- Hashed password in the database:

id	username	password_hash
1	admin	$2b$10$.ksz2xHGeWKQnuofMVIwb.uhAjN6r5F6FTL1zn8c6VMtRgeIcQIK2

4.3 Account Lockout and Brute Force Prevention

4.3.1 Account Lockout

This is an account-level defense to prevent brute-force attack on an account. If an account receives 5 - login requests consecutively with the wrong password, it will be locked in 5 minutes. Failed_login_atttempts and locked_until fields are used to conduct this feature. One to save the current number of failed attempts, one to save a specific time when the account is unlocked. An auto-reset mechanism is necessary to prevent permanent lockout.

- **Work flow:**

1. User enters wrong password
2. Increment failed_login_attempts by 1
3. If failed_login_attempts >= 5:
   + Set locked_until = current time + 5 minutes
   + Reset failed_login_attempts to 0
4. On next login attempt, check locked_until > current time
   + Yes: reject request immediately
   + No: proceed normally
5. After 5 minutes, locked_until expires → account automatically unlocked
6. On successful login: reset failed_login_attempts = 0, locked_until = null

- Increase fail_attempt if password invalid:

```
114         if (!passwordValid) {
115             const nextAttempts = Number(user.failed_login_attempts || 0) + 1;
116 
117             if (nextAttempts >= MAX_FAILED_ATTEMPTS) {
118                 const lockedUntil = new Date(
119                     Date.now() + LOCK_DURATION_MINUTES * 60 * 1000
120                 );
```

- Auto reset failed_login_attempts and locked_until:

```
102         if (user.locked_until && new Date(user.locked_until) <= new Date()) {
103             await user.update({
104                 failed_login_attempts: 0,
105                 locked_until: null,
106             });
107         }
```

4.3.2 Rate Limiting

Rate limiting is an IP-level defense - unlike Account Lockout which targets a specific account, rate limiting blocks by IP address. This covers the case where an attacker cycles through multiple usernames to avoid triggering account lockout. If more than 10 failed login requests are sent from the same IP within 10 minutes, the server responds with HTTP 429. skipSuccessfulRequests: true ensures legitimate users who login successfully are not penalized.

- **Workflow**:

1. Login request arrives from an IP
2. Check request count from that IP within the last 10 minutes
3. If count > 10:
    + Respond 429 Too Many Requests
    + Display "Too many attempts, try again after 10 minutes"
    + Stop processing
4. If count <= 10:
    + Pass request to next middleware
5. If login succeeds: request is NOT counted (skipSuccessfulRequests: true)
6. After 10-minute window resets: IP counter cleared automatically

```
import { rateLimit } from 'express-rate-limit';

export const loginLimiter = rateLimit({
    windowMs: 10 * 60 * 1000,
    limit: 10,
    standardHeaders: true,
    legacyHeaders: false,
    skipSuccessfulRequests: true,

    handler: (req, res) => {
        return res.status(429).render('login', {
            error: 'Quá nhiều lần đăng nhập thất bại từ thiết bị này. Vui lòng thử lại sau 10 phút.',
            redirect: '/dashboard',
        });
    },
});
```

4.4 CSRF Protection

4.4.1 Double-Submit Cookie Pattern

Standard CSRF protection uses a single token, but this project implements the Double-Submit Cookie pattern via csrf-csrf library. Each browser is assigned a unique bank_csrf_id cookie (UUID). The CSRF token is cryptographically bound to that browser ID - meaning a token generated for browser A cannot be used by browser B.

GET, HEAD, OPTIONS requests are excluded from validation since they do not modify server state.

- **Workflow**:

1. Browser visits login page (GET)
2. Server assigns unique bank_csrf_id cookie to browser
3. Server generates CSRF token bound to that browser ID
4. Token injected into form via hidden field _csrf
5. User submits form (POST)
6. Server extracts _csrf from request body
7. Server re-derives expected token using bank_csrf_id cookie
8. If match → proceed | If not → reject 403

```
const { generateCsrfToken, doubleCsrfProtection} = doubleCsrf({
    getSecret: () => process.env.CSRF_SECRET,
    getSessionIdentifier: (req) => req.cookies[CSRF_ID_COOKIE],
    cookieName: CSRF_TOKEN_COOKIE,
    cookieOptions,
    ignoredMethods: ['GET', 'HEAD', 'OPTIONS'],
    getCsrfTokenFromRequest: (req) => req.body._csrf,
});

export { doubleCsrfProtection };
```

4.4.2 CSRF Context Rotation

After a successful login or logout, rotateCsrfContext() is called to invalidate the previous CSRF context. A new browser ID is issued and the old token cookie is cleared. This prevents an attacker who obtained a pre-login CSRF token from reusing it in a post-login request.

- **Worksflow**:

1. User logs in successfully
2. rotateCsrfContext() called
3. New UUID generated → overwrites bank_csrf_id cookie
4. Old bank_csrf_token cookie cleared
5. Next form request receives a fresh token bound to new ID
6. Same process on logout

```
69  export const rotateCsrfContext = (req, res) => {
70      const newCsrfId = randomUUID();
71      res.cookie(CSRF_ID_COOKIE, newCsrfId, cookieOptions);
72      res.clearCookie(CSRF_TOKEN_COOKIE, cookieOptions);
73  };
```

4.5 Role-Based Access Control

4.5.1 requireAuth Middleware

requireAuth verifies that every request to a protected route carries a valid JWT. The token is read from the HttpOnly cookie - inaccessible to JavaScript - and verified against JWT_SECRET. If the token is missing or expired, the user is redirected to the login page with the original URL preserved in the ?redirect= parameter

- **Workflow**:

1. Request arrives at protected route
2. Extract token from req.cookies.token
3. If no token → redirect */login?redirect=<original URL>*
4. Else jwt.verify(token, JWT_SECRET)
5. If invalid or expired → clear cookie → redirect */login*
6. If valid → attach req.user = { id, username, role } → next()

```
import jwt from 'jsonwebtoken';

export const requireAuth = (req, res, next) => {
    const token = req.cookies?.token;
    if (!token) return res.redirect(`/login?redirect=${req.originalUrl}`);
    try {
        req.user = jwt.verify(token, process.env.JWT_SECRET);
        next();
    } catch {
        res.clearCookie('token');
        res.redirect(`/login?redirect=${req.originalUrl}`);
    }
};
```

4.5.2 requireRole Middleware

requireRole enforces authorization after authentication - it checks whether the authenticated user's role matches the required role for the requested route. Any unauthorized access attempt is logged to the audit table before returning 403.

- **Workflow**:

1. req.user already attached by requireAuth
2. Check req.user.role against allowed roles
3. If not allowed:
    + AuditLog UNAUTHORIZED_ACCESS with IP + attempted URL
    + Return 403 Forbidden
4. If allowed → next()

```
import AuditLog from '../model/audit.model.js';

export const requireRole = (...roles) => async (req, res, next) => {
    if (!roles.includes(req.user?.role)) {
        await AuditLog.create({
            event_type: 'UNAUTHORIZED_ACCESS',
            username:   req.user?.username || 'unknown',
            ip_address: req.ip,
            details:    `Tried to access ${req.originalUrl} with role ${req.user?.role}`,
        });
        return res.status(403).render('error', { message: 'Forbidden – không có quyền truy cập' });
    }
    next();
};
```

4.6 SQL Injection - Vulnerable Module

4.6.1 String Concatenation Query

The vulnerable module uses mysql2 with raw string concatenation - user input is inserted directly into the SQL string without any escaping or parameterization. This is the root cause of all SQL Injection vulnerabilities demonstrated in this module.

- Vulnerable login:

```
29
30    const sql = `SELECT * FROM users WHERE username='${username}' AND password_hash='${password}'`;
31
```

- Vulnerable search:

```
73
74    const sql = `SELECT id, username, role FROM users WHERE username LIKE '%${name}%'`;
75
```

- Vulnerable search by ID:

```
207
208    const sql = `SELECT id, username, role FROM users WHERE id = ${user_id}`;
```

4.6.2 Attack Scenarios Demonstrated

There are four main scenarios demonstrated:

Scenario	Form	Sample Payload	Effect
Auth Bypass	Vulnerable Login	admin'#	Comment out password check
OR Injection	Vulnerable Login	' OR '1'='1'#	Return all users
UNION - LIKE	Vulnerable Search	' UNION SELECT 1,password_hash,3 FROM users#	Leak password hashes
UNION - ID	Vulnerable Search by ID	1 OR 1=1#	No quotes needed - numeric field
Time-based Blind	Blind Injection	alice' AND SLEEP(3)#	Confirm injection via response delay

4.6.3 Injection Detection

Even on vulnerable endpoints, the application detects and logs suspicious input patterns using a regex check before executing the query. This does not prevent the attack - the query still executes - but ensures every injection attempt is recorded in the audit log.

- Detect sql injection pattern:

```
7   const containsSqlInjectionPattern = (value = '') => {
8       const pattern = /('|\bOR\b|\bUNION\b|\bSELECT\b|--|#|;)/i;
9       return pattern.test(value);
```

- Audit log:

```
37              await AuditLog.create({
38                  event_type: 'SQL_INJECTION_ATTEMPT',
39                  username: username || 'anonymous',
40                  ip_address: req.ip,
41                  details: 'Suspicious payload detected in vulnerable login demo',
42              });
```

4.7 Secure Query Handling

4.7.1 Sequelize ORM Parameterization

Sequelize never concatenates user input into SQL strings. Input is passed as a bound parameter - the database driver handles escaping internally, making injection structurally impossible regardless of the payload content.

- Secure login:

```
119         const user = await User.findOne({
120             where: { username },
121         });
```

- Secure search:

```
176         const users = await User.findAll({
177             attributes: ['id', 'username', 'role'],
178             where: {
179                 username: {
180                     [Op.like]: `%${name}%`,
181                 },
182             },
183         });
```

4.7.2 BCrypt Separation from Query

In the vulnerable login, password is checked inside the SQL WHERE clause - meaning a crafted payload can bypass it entirely. The secure version removes password from the query completely. Sequelize fetches the user by username only, then BCrypt performs the comparison separately in application code.

- Vulnerable password check:

```
30      const sql = `SELECT * FROM users WHERE username='${username}' AND password_hash='${password}'`;
31
```

- Secure password check:

```
119         const user = await User.findOne({
120             where: { username },
121         });
122
123         const passwordValid =
124             user && bcrypt.compareSync(password, user.password_hash);
```

4.7.3 Vulnerable vs Secure Comparison

Table 4.1: Vulnerable vs Secure Query Comparison

Aspect	Vulnerable	Secure
Query method	String concatenation	Parameterized / ORM
Input handling	Directly embedded	Escaped and bound
Injection risk	High	None
Password check	In SQL WHERE clause	BCrypt compare (separate)
DB connection	app_vulnerable (ALL PRIVILEGES)	app_admin (limited)
Detection	Regex log only	N/A - structurally prevented

4.8 Least-Privilege Database Access

```
15  -- CUSTOMER: chỉ xem/thao tác data của mình
16  -- Không được đọc bảng users (thông tin nhạy cảm)
17
18  GRANT SELECT, INSERT ON banking.transactions TO 'app_customer'@'localhost';
19  GRANT SELECT, UPDATE ON banking.accounts     TO 'app_customer'@'localhost';
20
21  -- AUDITOR: read-only toàn bộ, không được sửa gì
22
23  GRANT SELECT ON banking.users        TO 'app_auditor'@'localhost';
24  GRANT SELECT ON banking.accounts     TO 'app_auditor'@'localhost';
25  GRANT SELECT ON banking.transactions TO 'app_auditor'@'localhost';
26  GRANT SELECT ON banking.audit_logs   TO 'app_auditor'@'localhost';
27
28  -- ADMIN: quản lý users và accounts, không chuyển tiền
29
30  GRANT SELECT, INSERT, UPDATE, DELETE ON banking.users      TO 'app_admin'@'localhost';
31  GRANT SELECT, INSERT, UPDATE         ON banking.accounts   TO 'app_admin'@'localhost';
32  GRANT SELECT                         ON banking.audit_logs TO 'app_admin'@'localhost';
33  💡
34  -- VULNERABLE: full quyền — dùng cho demo tấn công
35
36  GRANT ALL PRIVILEGES ON banking.* TO 'app_vulnerable'@'localhost';
37
```

4.9 Security Headers

4.9.1 Helmet Middleware

The application uses the helmet middleware to automatically attach a set of HTTP security headers to every response. These headers instruct the browser on how to handle content, preventing a class of client-side attacks without any changes to application logic.

```
app.use(helmet({
    contentSecurityPolicy: {
        directives: {
            defaultSrc: ["'self'"], styleSrc: ["'self'", "'unsafe-inline'"],
            scriptSrc: ["'self'"], imgSrc: ["'self'", 'data:'],
            objectSrc: ["'none'"], baseUri: ["'self'"],
            formAction: ["'self'"], frameAncestors: ["'none'"],
        },
    },
    strictTransportSecurity: false,
}));
```

4.9.2 Content Security Policy

CSP is the most significant header configured. It restricts what resources the browser is allowed to load, limiting the blast radius of an XSS attack even if one occurs.

Header	Value	Protection
default-src	'self'	Only load resources from same origin
script-src	'self'	Block inline scripts and external JS
style-src	'self' 'unsafe-inline'	Allow inline CSS (EJS limitation)
img-src	'self' data:	Block external image loading
object-src	'none'	Block Flash and plugins
form-action	'self'	Forms can only submit to same origin
frame-ancestors	'none'	Block iframe embedding (Clickjacking)
base-uri	'self'	Prevent base tag hijacking
HSTS	Disabled	HTTP in dev - enable on production HTTPS

4.10 Audit Logging

Table 4.2: Audit Log Schema

Field	Type	Description
id	INT AUTO_INCREMENT	Primary key
user_id	INT (nullable)	FK to users - null for anonymous attempts
event_type	VARCHAR(255)	Category of the security event
username	VARCHAR(255)	Snapshot of username at time of event
ip_address	VARCHAR(255)	IP address of the request
details	TEXT	Additional context about the event
createdAt	DATETIME	Timestamp auto-set by Sequelize

Table 4.3: Event Types

Event Type	**Trigger**	**Source**
LOGIN_SUCCESS	Correct credentials	auth.controller.js
LOGIN_FAIL	Wrong password or username not found	auth.controller.js
LOGIN_BLOCKED	Account locked (admin disabled or temp lock)	auth.controller.js
LOGOUT	User logs out	auth.controller.js
ACCOUNT_TEMPORARILY_LOCKED	5 consecutive failed attempts	auth.controller.js
UNAUTHORIZED_ACCESS	Role mismatch on protected route	rbac.middleware.js
TRANSFER	Successful fund transfer	transfer.controller.js
SQL_INJECTION_ATTEMPT	Injection pattern detected in input	vulnerable.controller.js
SECURE_LOGIN_SUCCESS	Valid credentials on secure demo form	vulnerable.controller.js
SECURE_LOGIN_FAIL	Invalid credentials on secure demo form	vulnerable.controller.js

Chapter 5. Security Testing and Results

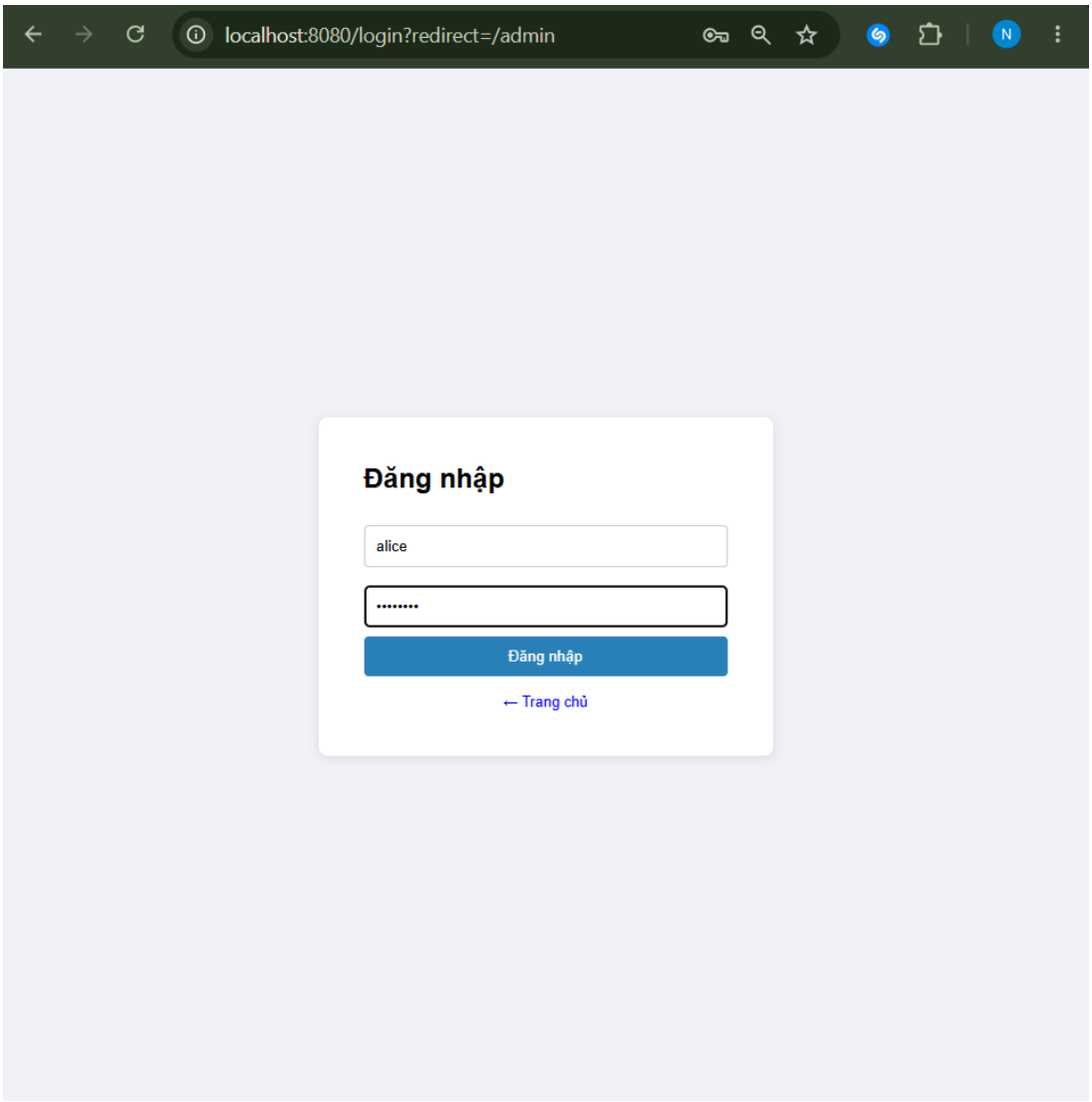
5.1 Testing Methodology

The security testing was conducted manually using a web browser and MySQL Workbench. Black-box testing was applied to evaluate the application from the perspective of different user roles, including CUSTOMER, ADMIN, and AUDITOR. SQL Injection payloads were submitted through the vulnerable login and search forms to verify whether authentication bypass and data extraction were possible. The same payloads were then tested against the secure forms to confirm that Sequelize ORM and BCrypt-based authentication prevented the attack. In addition, account lockout, rate limiting, CSRF protection, RBAC, audit logging, and least-privilege database access were verified using screenshots and database query outputs.

5.2 RBAC Test Results

Table 5.1: RBAC Test Cases

#	Role / Account	Action	Expected Output	Result
1.1	CUSTOMER (alice)	Access /admin	403 Forbidden	Deny
1.2	CUSTOMER (alice)	Access /audit	403 Forbidden	Deny
1.3	AUDITOR (auditor)	Access /transfer	403 Forbidden	Deny
1.4	ADMIN (admin)	Access /admin	User list displayed	Pass
1.5	AUDITOR (auditor)	Access /audit	Audit logs displayed	Pass


localhost:8080/login?redirect=/admin
Đăng nhập
alice
Đăng nhập
← Trang chủ

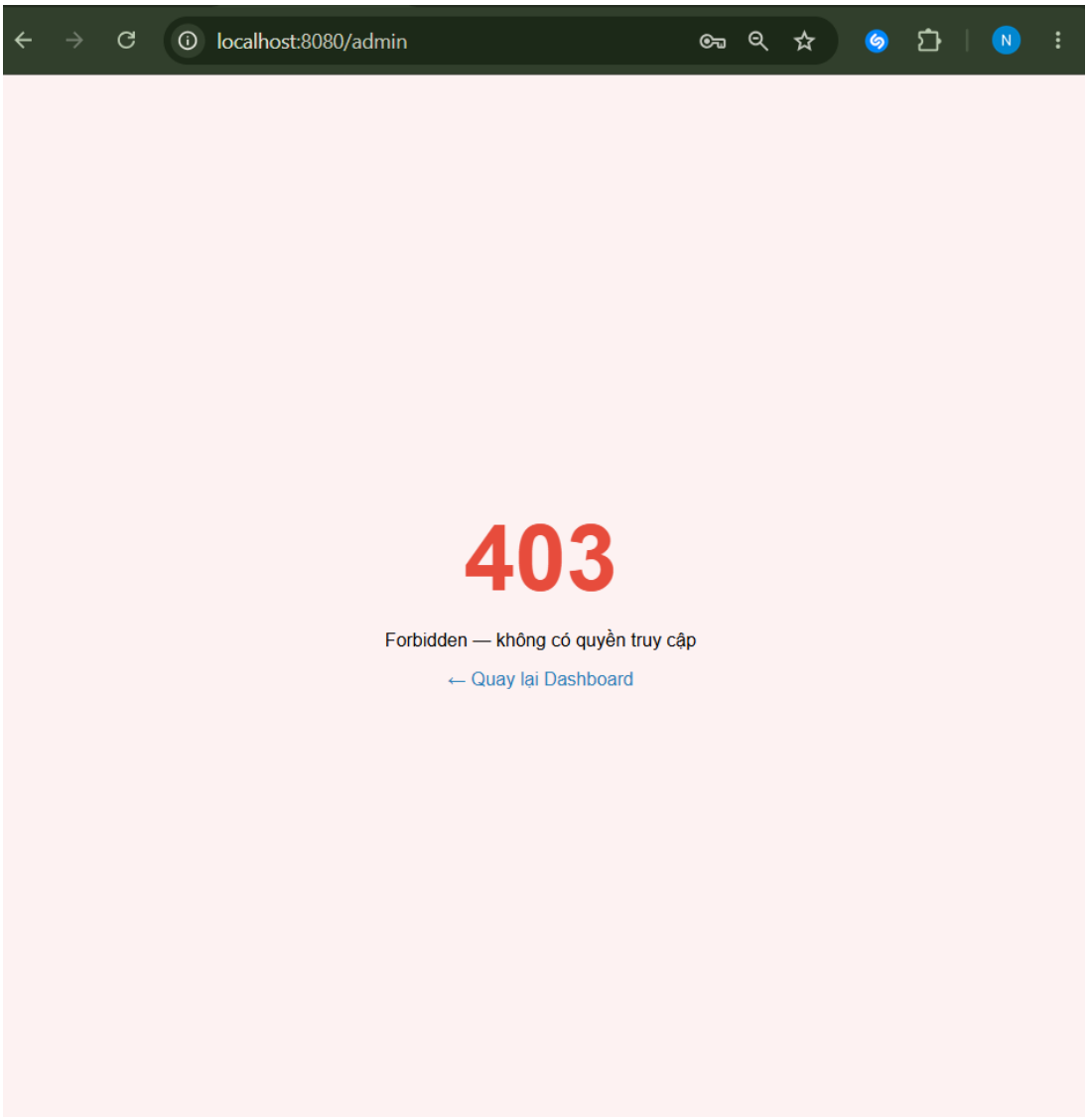
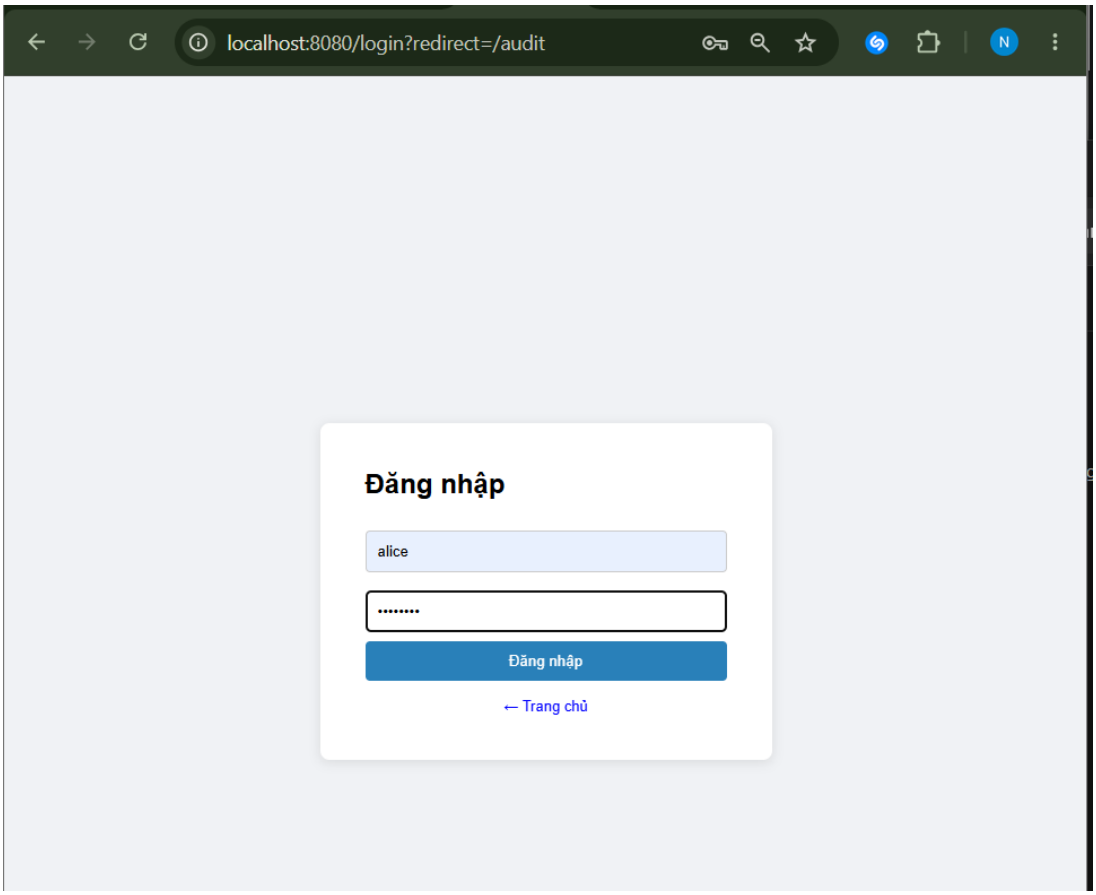


Figure 5.1: Customer is denied access to the Admin Panel.

The CUSTOMER account attempted to access the Admin Panel. The system denied the request because the route is protected by the ADMIN role requirement. This confirms that customers cannot access administrative functions.



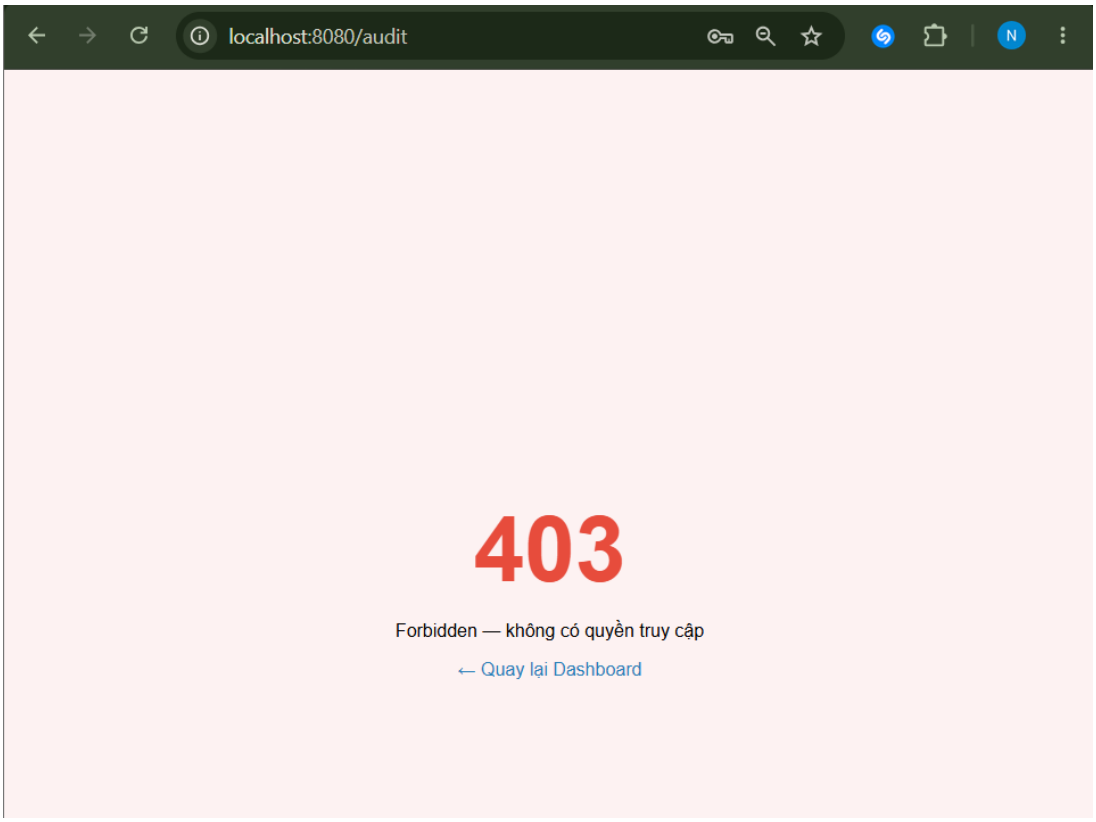
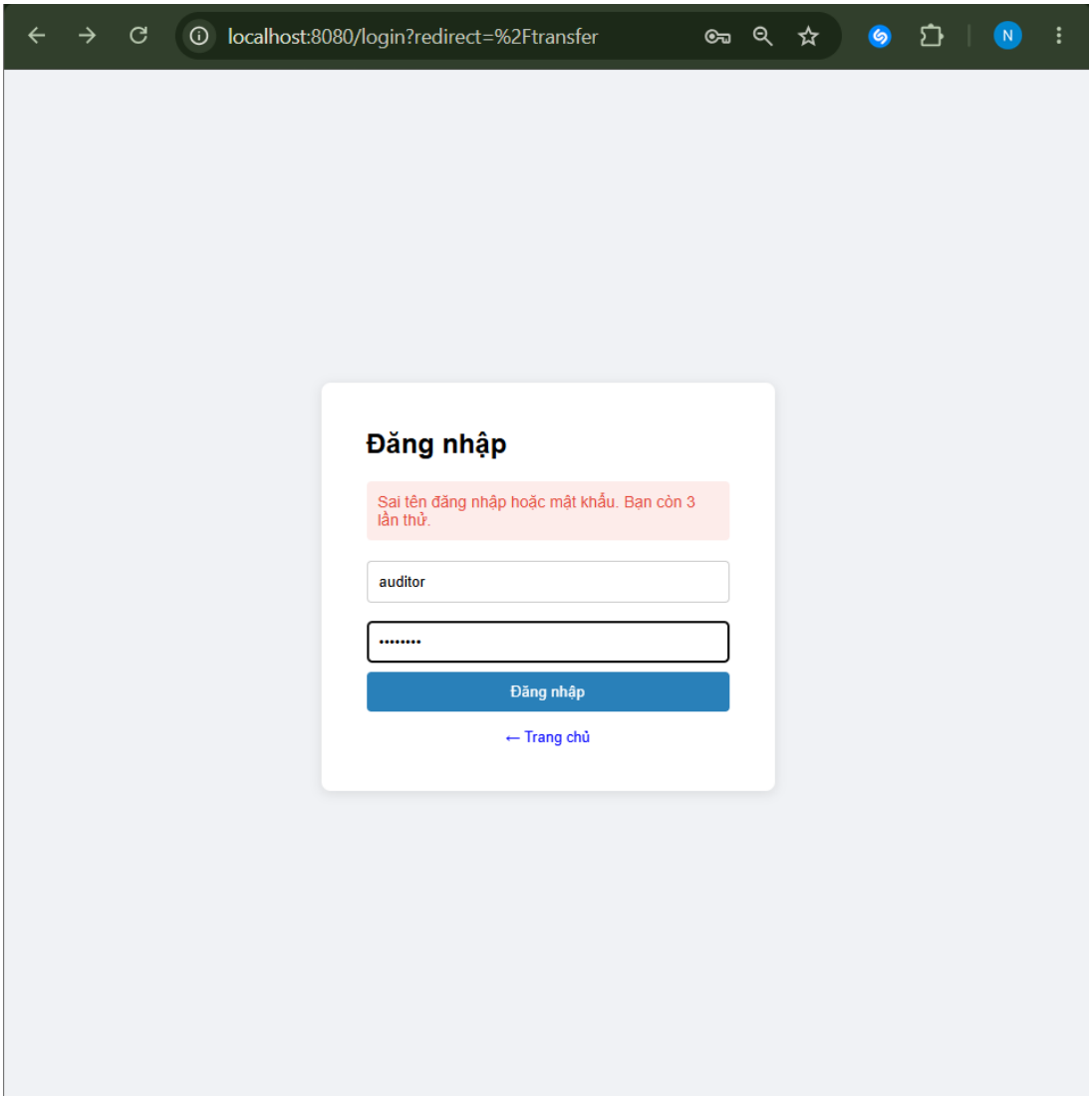


Figure 5.2: Customer is denied access to Audit Logs.

The CUSTOMER role was not allowed to access the audit log page. This verifies that security monitoring data is restricted to authorized roles only.



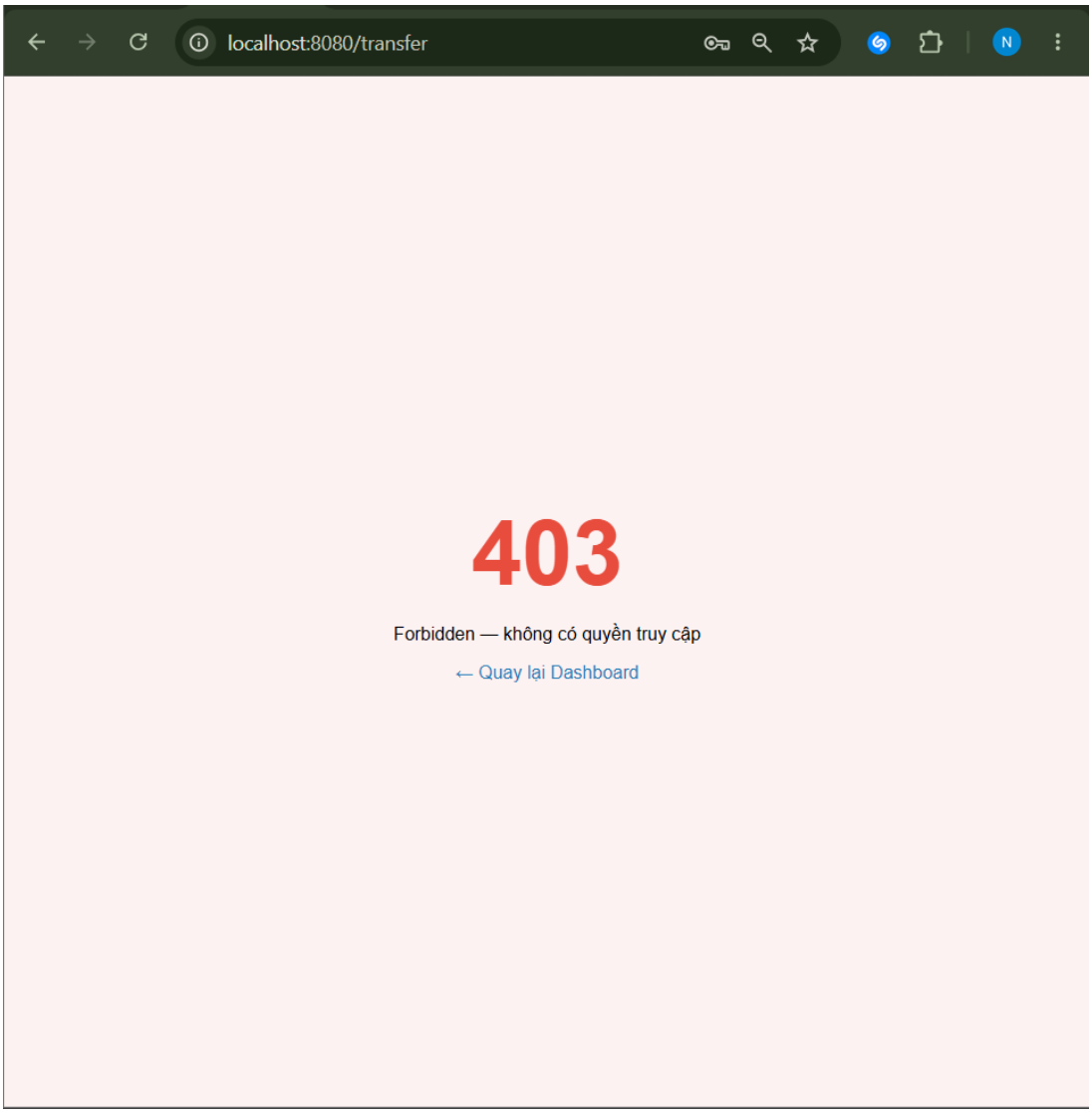
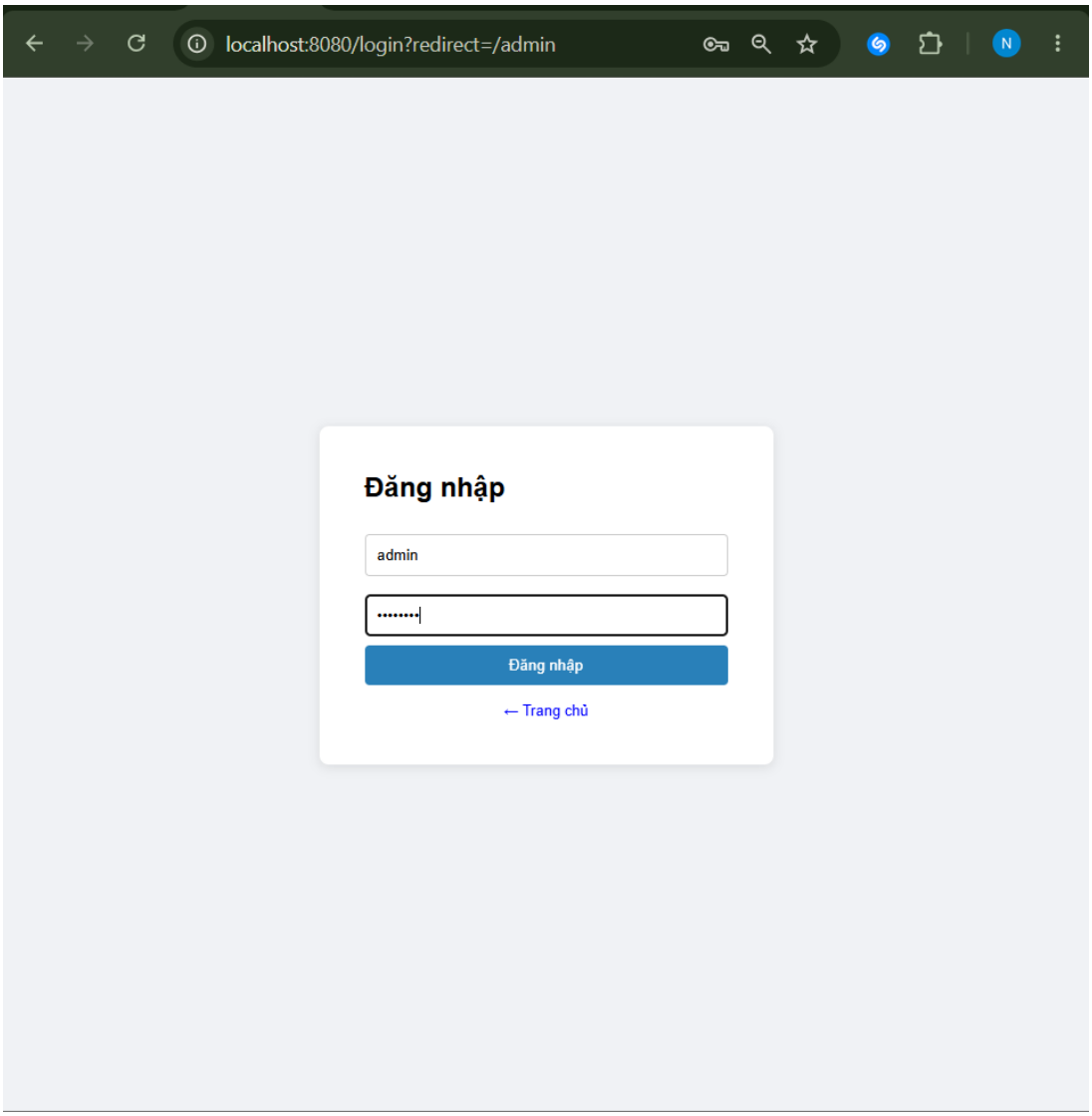


Figure 5.3: Auditor is denied access to the Transfer function.

The AUDITOR role attempted to access the money transfer page. The system denied the request because only CUSTOMER accounts are allowed to perform transfers.


localhost:8080/login?redirect=/admin
Đăng nhập
admin
Đăng nhập
← Trang chủ

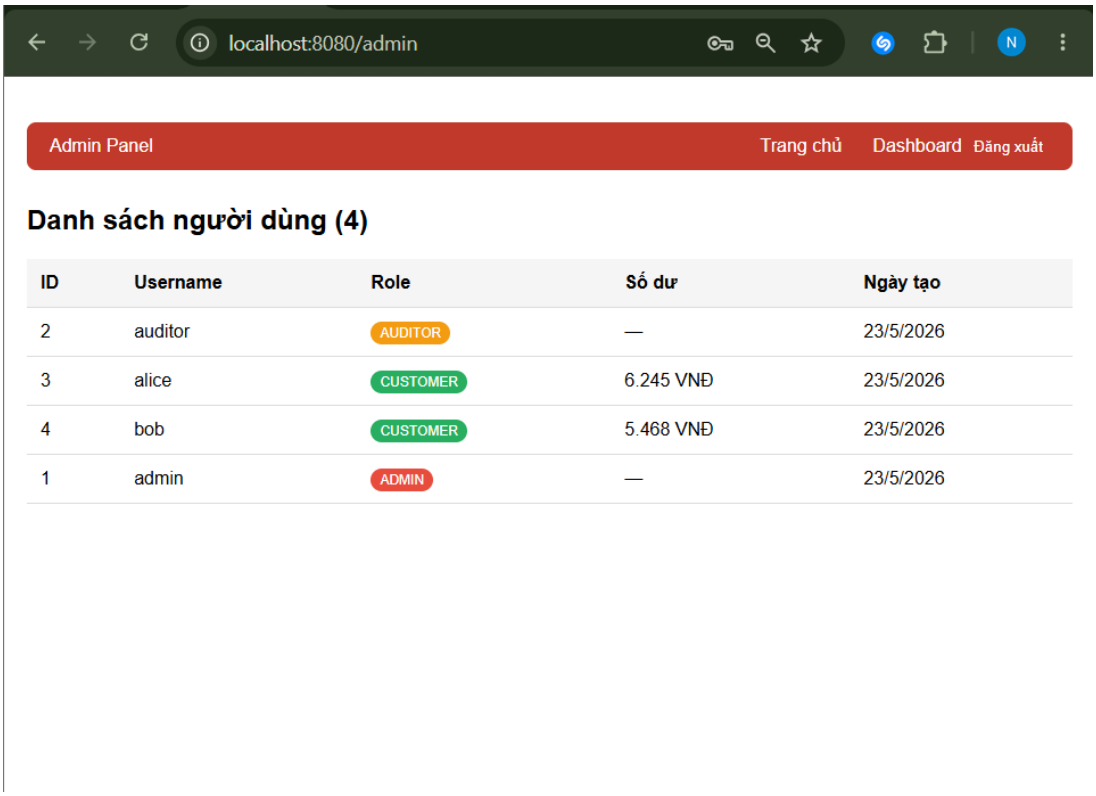
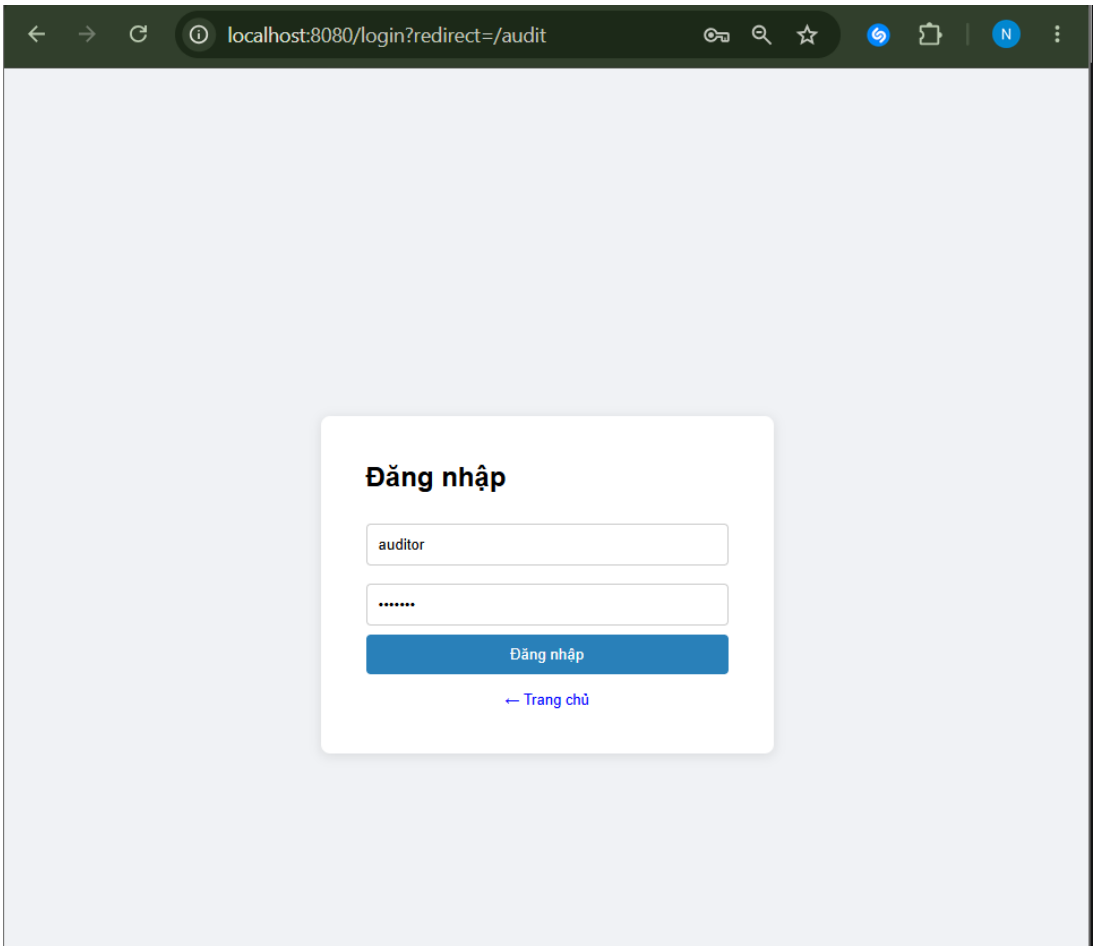


Figure 5.4: Admin successfully accesses the Admin Panel.

The ADMIN account successfully accessed the Admin Panel and viewed the user list. This confirms that administrative permissions are correctly granted to the ADMIN role.


localhost:8080/login?redirect=/audit
Đăng nhập
auditor
Đăng nhập
← Trang chủ

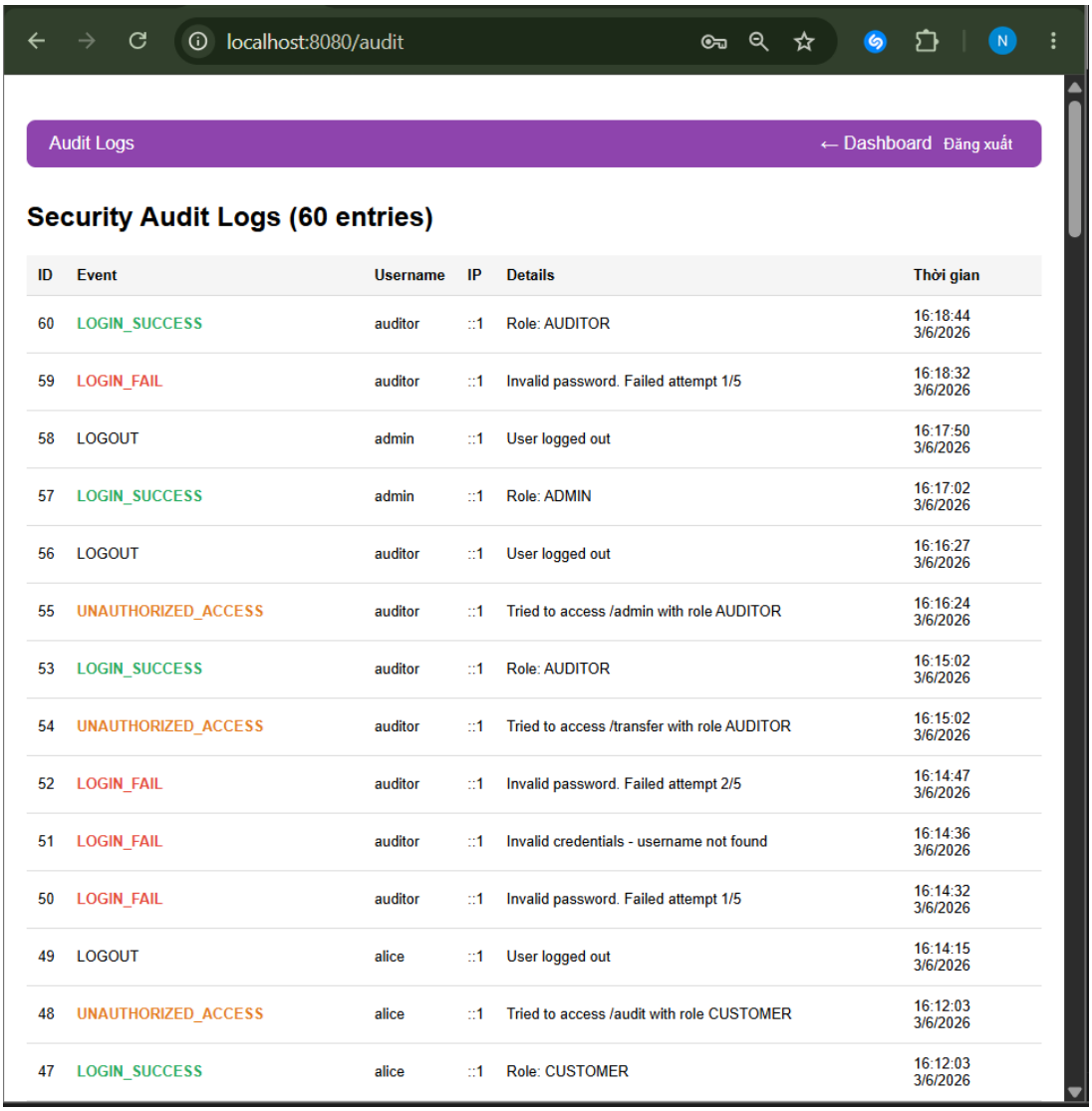


Figure 5.5: Auditor successfully accesses Audit Logs.

The AUDITOR account successfully accessed the Audit Logs page. This confirms that auditors can review security events but cannot perform banking transactions or administrative actions.

5.3 Account Lockout and Rate Limiting Test Results

Table 5.2: Brute Force Prevention Test Cases

#	Action	Expected Output	Result
2.1	Wrong password 5 times (alice)	Account locked for 5 minutes	Pass
2.2	Login after lock expires	Login succeeds	Pass
2.3	10+ POST /login in 10 min	HTTP 429 Too Many Requests	Pass

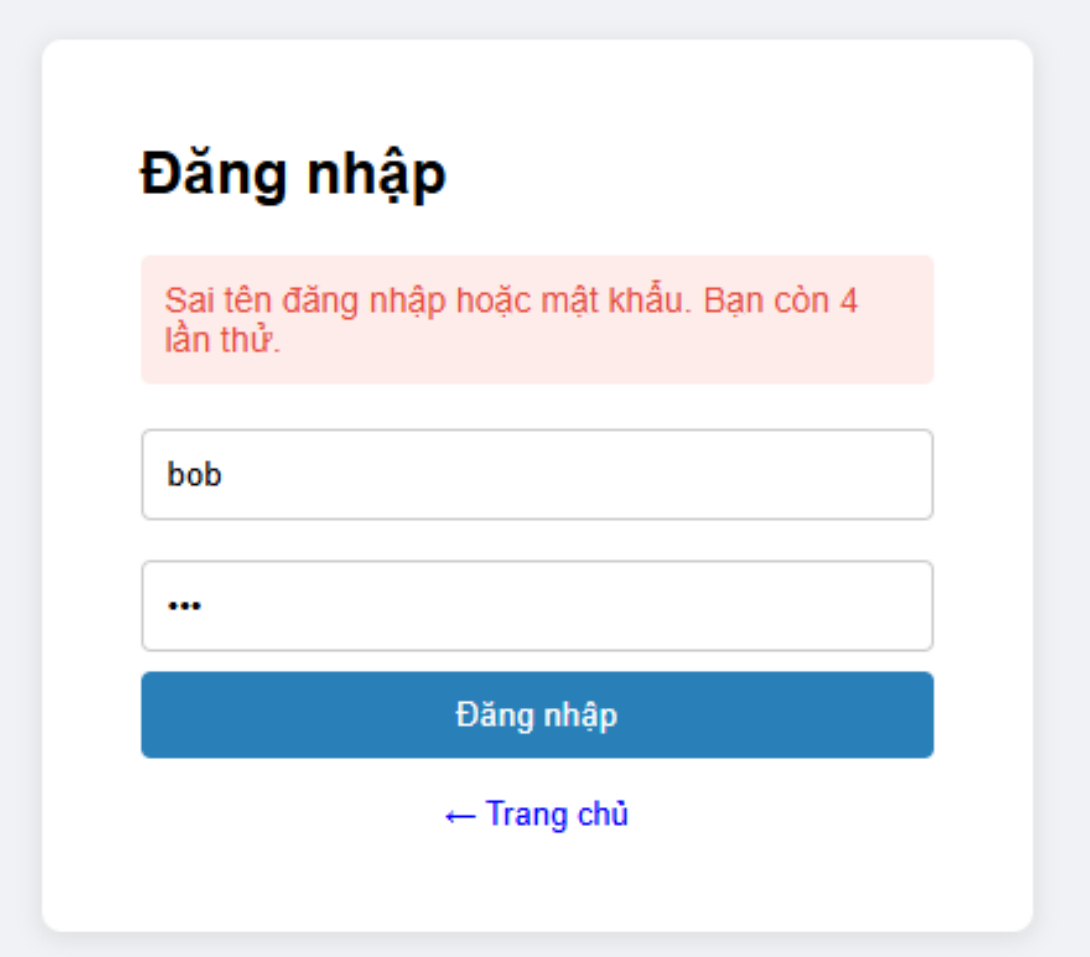


Figure 5.6: Failed login attempt is counted by the system.

The system detected an invalid password and increased the failed login counter for the account. The remaining attempt message shows that account-level brute-force protection is active.

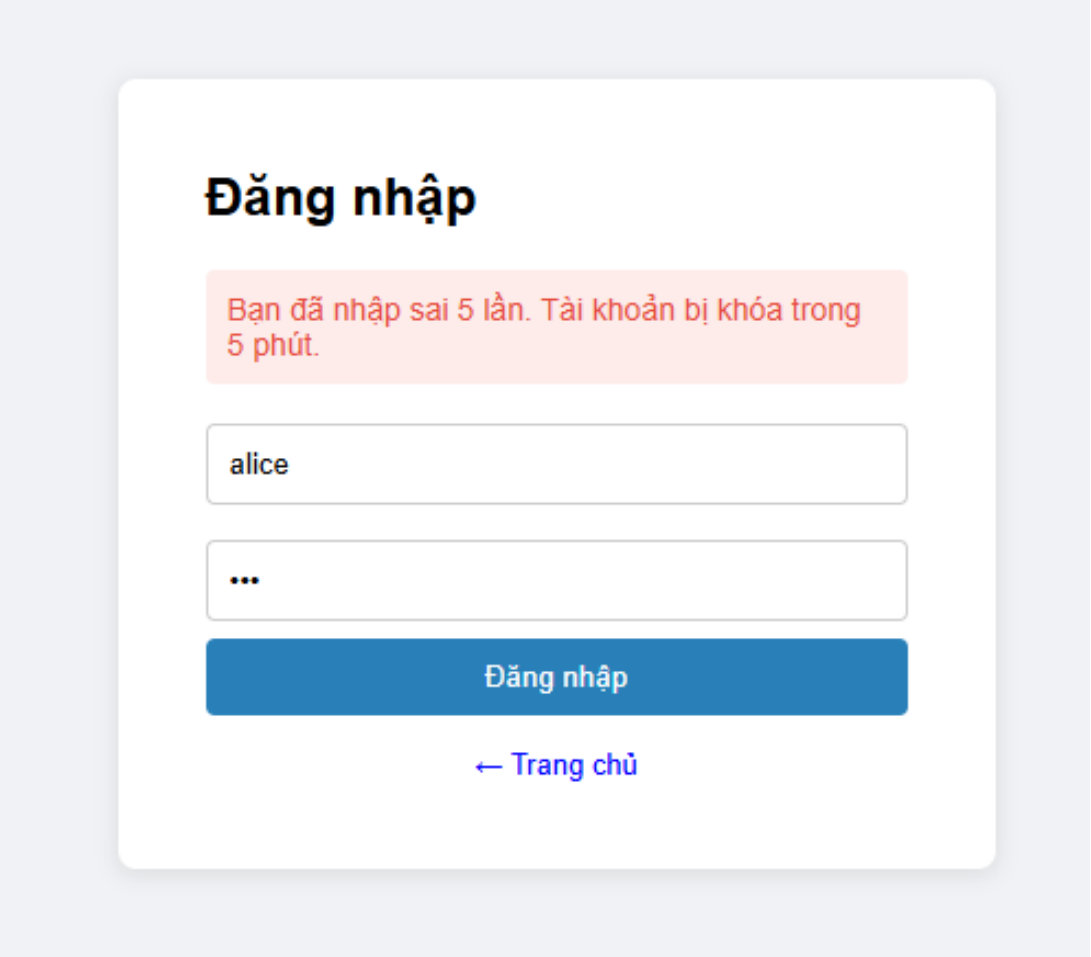


Figure 5.7: Account is temporarily locked after five failed login attempts.

After five consecutive failed login attempts, the account was temporarily locked for five minutes. This confirms that the lockout policy prevents repeated password guessing against a single account.

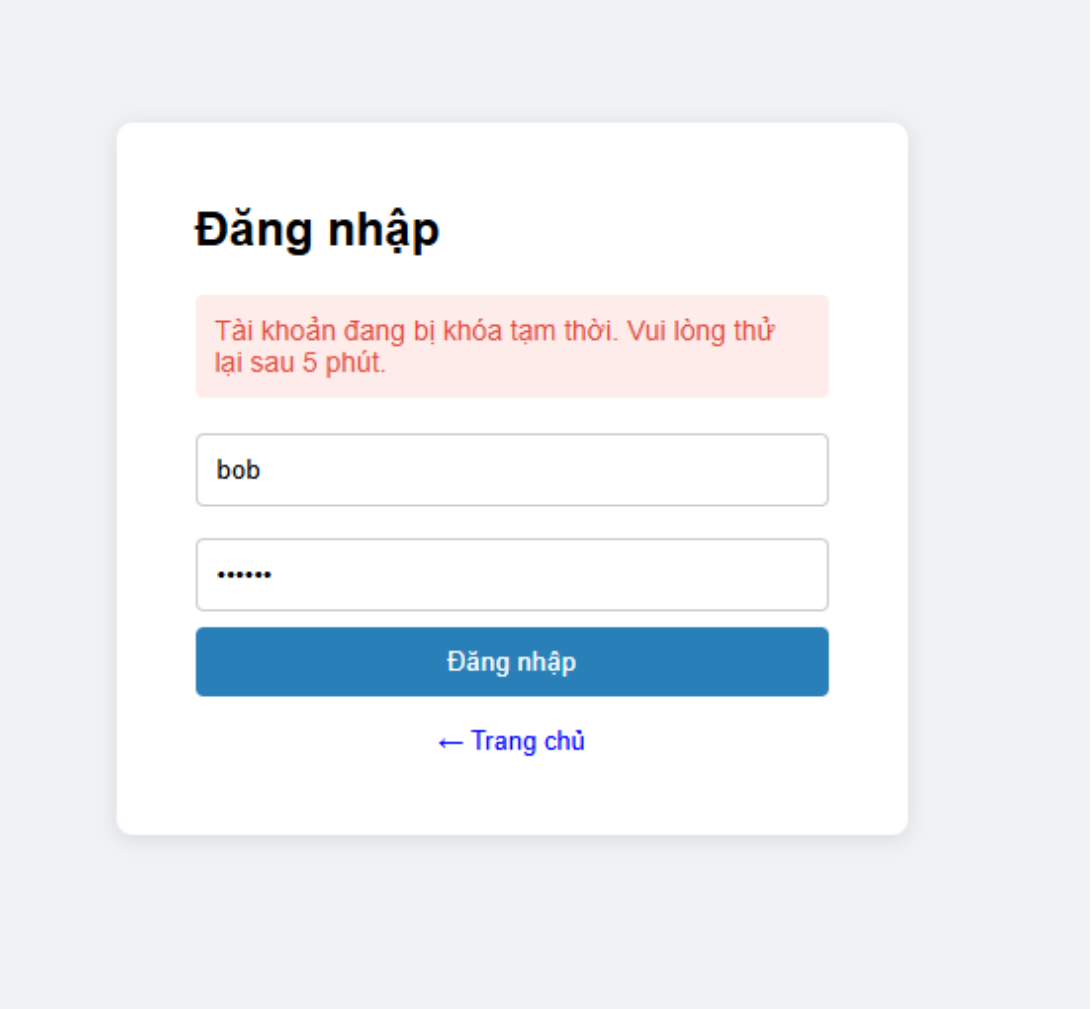


Figure 5.8: Login is blocked while the temporary lockout is active.

Even with the correct password, the user could not log in during the lockout period. This proves that the locked_until value is enforced before password verification.

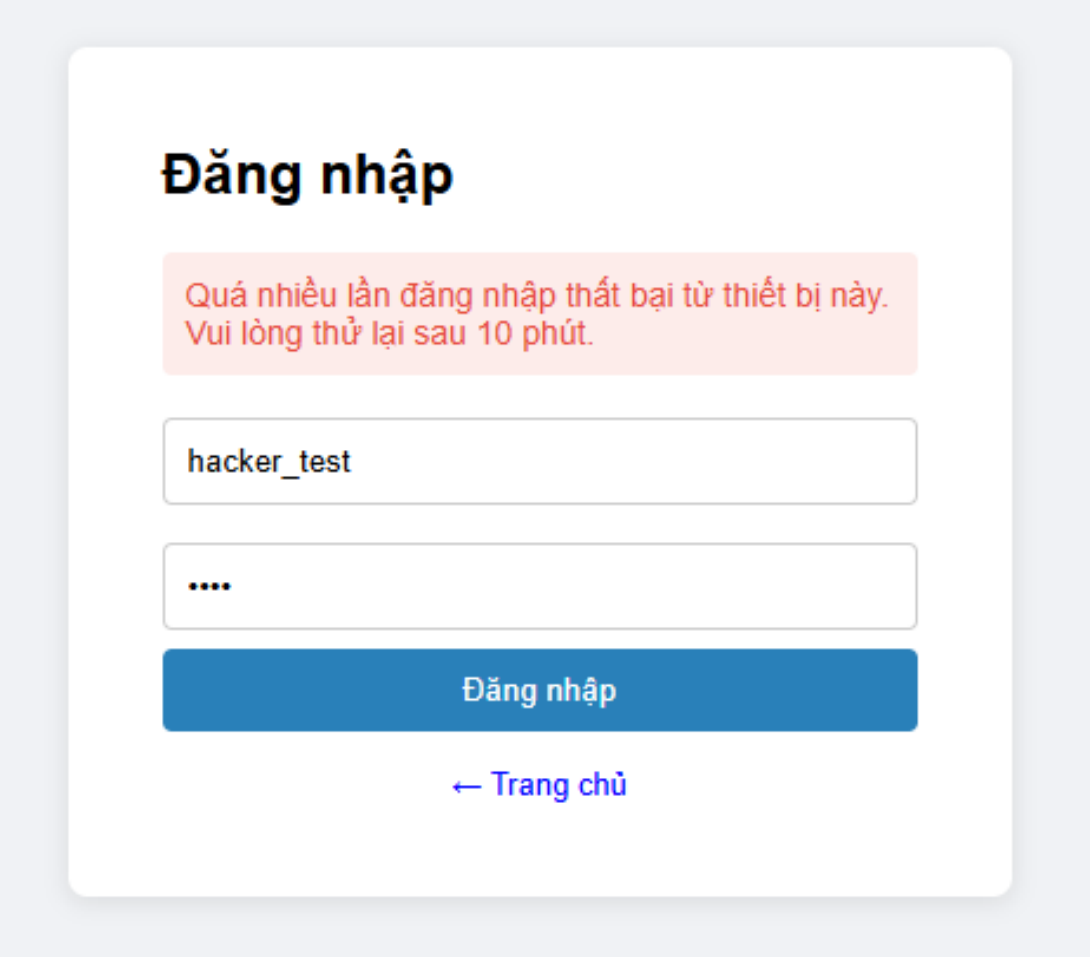


Figure 5.9: Rate limiting blocks excessive login attempts from the same IP address.

The rate limiter blocked repeated failed login requests from the same client. This complements account lockout by stopping brute-force attempts that rotate multiple usernames.

5.4 SQL Injection Test Results

5.4.1 Authentication Bypass

Table 5.3: Authentication Bypass Test Cases

#	Payload	Expected	Result
3.1	admin'#	Return admin without password	Pass
3.2	' OR '1'='1'#	Return all users	Pass
3.3	Same payloads on Secure Login	No result returned	Pass

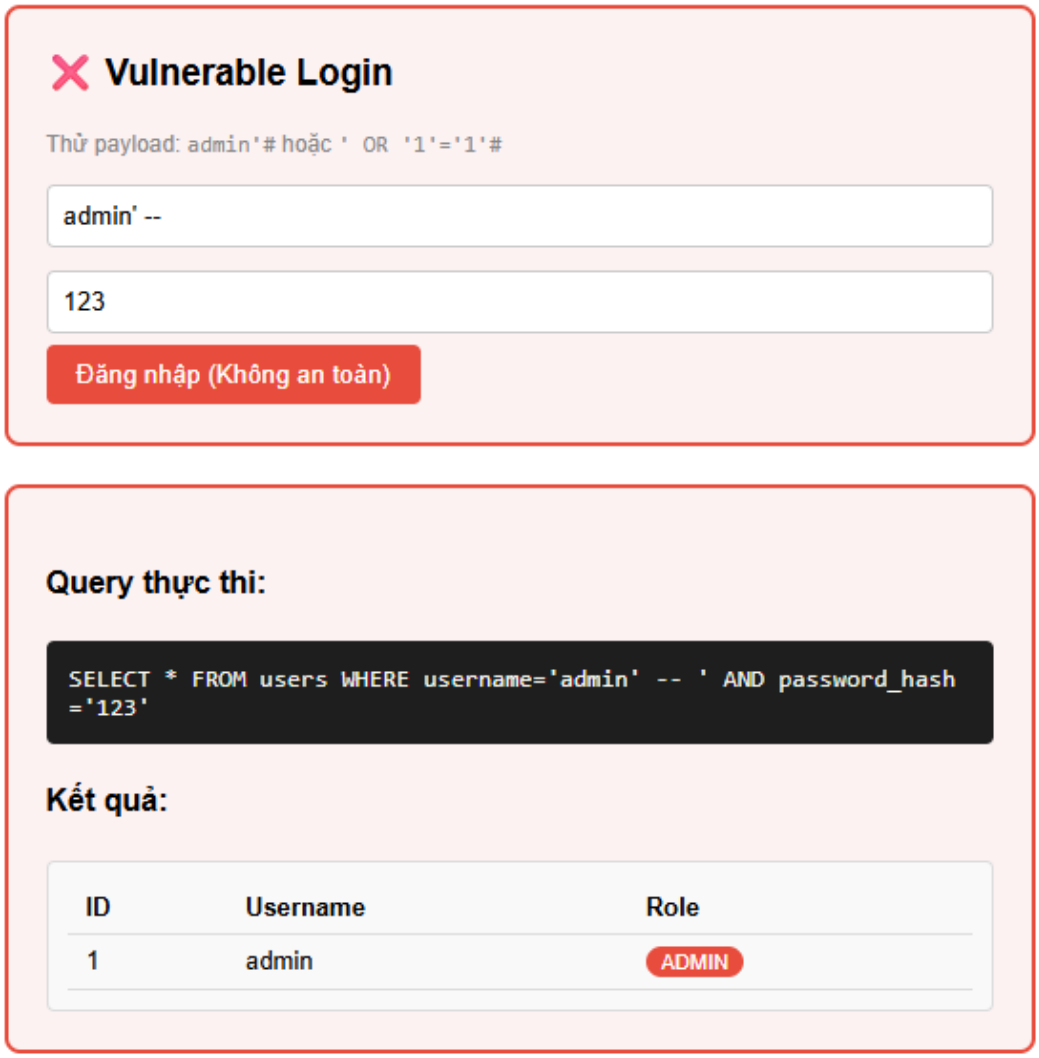


Figure 5.10: Authentication bypass using SQL Injection in the vulnerable login form.

The vulnerable login form directly concatenated user input into the SQL query. The payload admin' - commented out the password condition and returned the admin account without requiring the correct password.

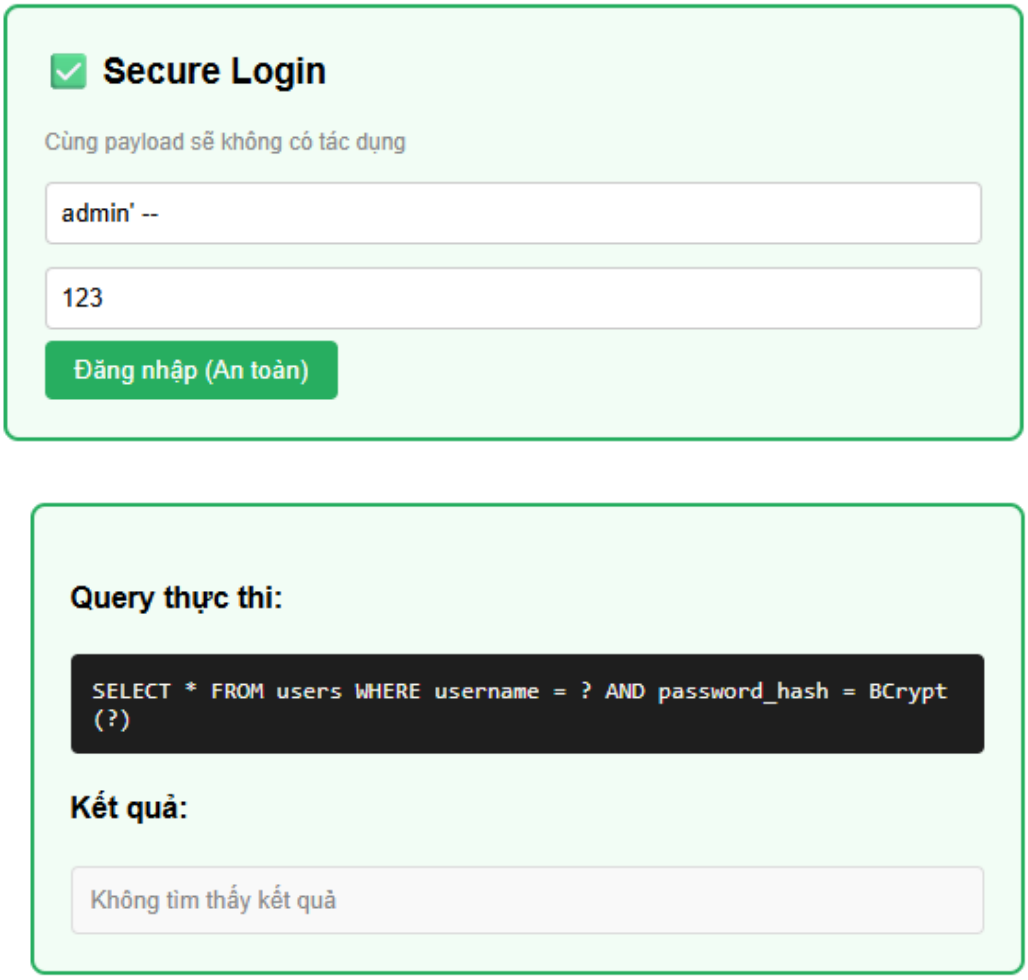


Figure 5.11: Secure login blocks the same SQL Injection payload.

The same payload failed against the secure login form. The secure implementation retrieves users through Sequelize and verifies the password separately with BCrypt, so the injected SQL syntax is treated as normal input.

5.4.2 UNION-Based Data Extraction

Table 5.4: UNION Injection Test Cases

#	Payload	Expected	Result
3.4	' UNION SELECT 1,table_name,'CUSTOMER' FROM information_schema.tables WHERE table_schema='banking'#	Table names are leaked	Pass
3.5	' UNION SELECT 1,password_hash,'CUSTOMER' FROM users#	Password hashes are leaked	Pass
3.6	Same UNION payloads on Secure Search	No sensitive data returned	Pass

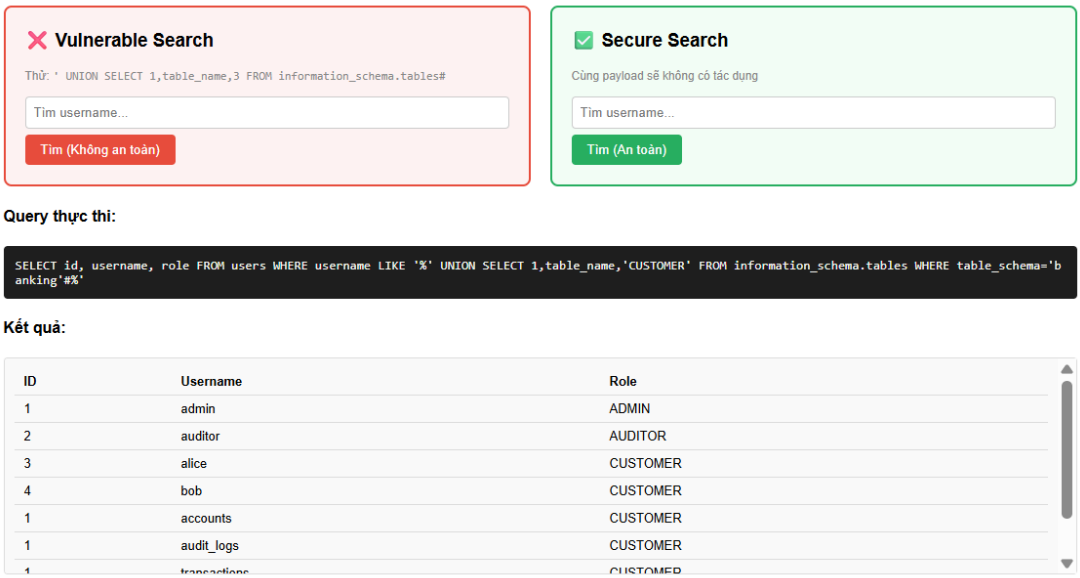


Figure 5.12: UNION-based SQL Injection extracts table names.

The UNION-based payload extracted table names from information_schema.tables through the vulnerable search form. This demonstrates that SQL Injection can be used for database reconnaissance, not only authentication bypass.

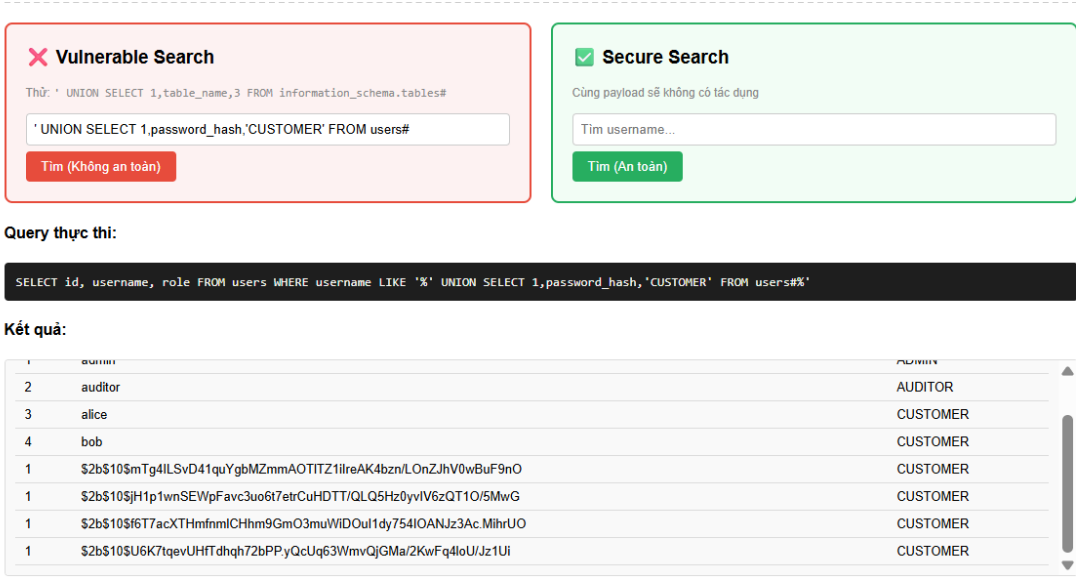


Figure 5.13: UNION-based SQL Injection extracts password hashes.

The vulnerable search form also allowed the attacker to extract BCrypt password hashes from the users table. Although BCrypt hashes are not plaintext passwords, exposing them is still a serious security risk because attackers may attempt offline password cracking.

5.4.3 Secure Search Verification

Table 5.5: Search by ID Injection Test Cases

#	Payload	Expected	Result
3.7	’ UNION SELECT 1,table_name,’CUSTOMER’ FROM information_schema.tables WHERE table_schema=’banking’#	No table names leaked	Pass
3.8	’ UNION SELECT 1,password_hash,’CUSTOMER’ FROM users#	No password hashes leaked	Pass

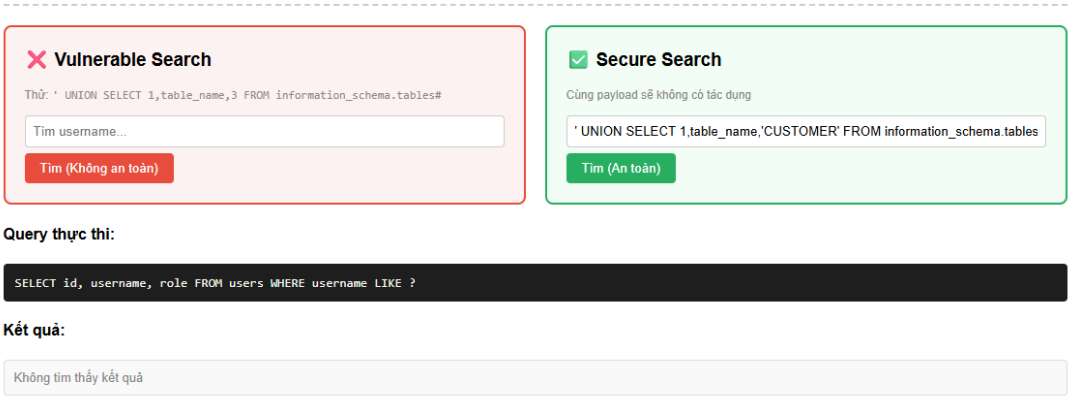


Figure 5.14: Secure search blocks UNION-based SQL Injection payload.

The app_customer database account was not allowed to read sensitive user credentials. This confirms that least-privilege database access limits damage even if a lower-privileged database account is compromised.

5.5 Least-Privilege Test Results

Table 5.6: Least-Privilege Comparison

DB User	Privileges	Test Query	Result	Outcome
app_vulnerable	ALL PRIVILEGES on banking.*	UNION SELECT from information_schema	Table names leaked	Attack succeeds
app_customer	Limited access to accounts and transactions	SELECT username, password_hash FROM users	Access denied	Attack mitigated
Secure query	ORM / parameterized query	Same injection payload	No sensitive result	Attack blocked

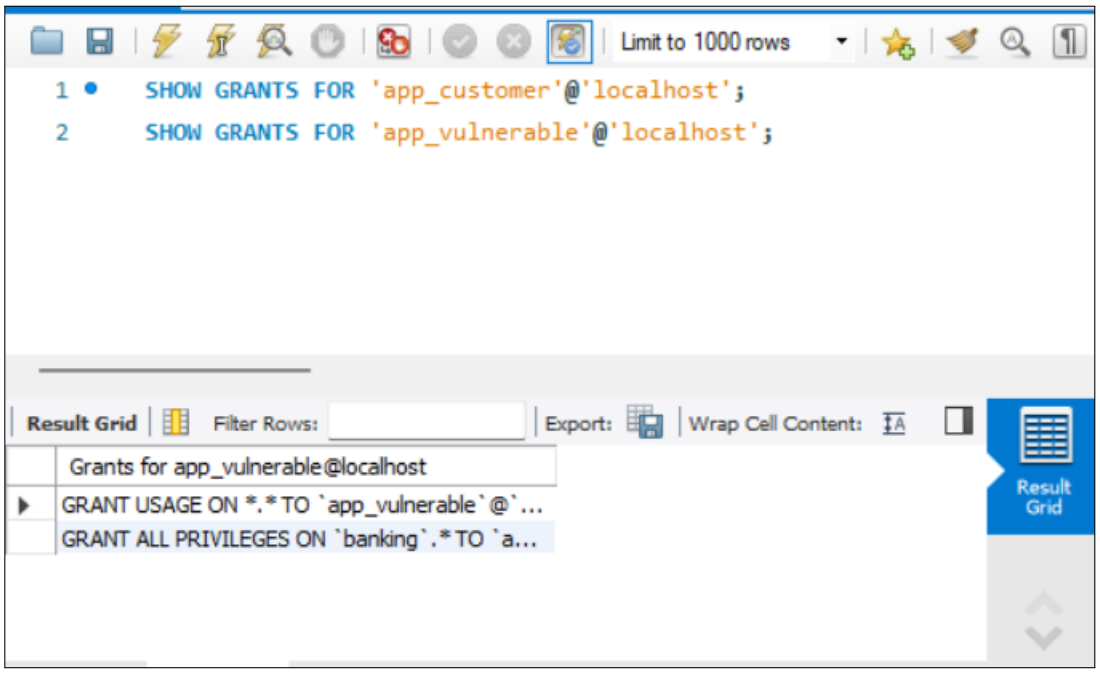


Figure 5.15: MySQL grants assigned to restricted and vulnerable database users.

The SHOW GRANTS output shows that app_customer was granted only limited permissions on accounts and transactions, while app_vulnerable was intentionally granted broad privileges for demonstration. This difference shows how database-level privileges can limit or increase the impact of SQL Injection.

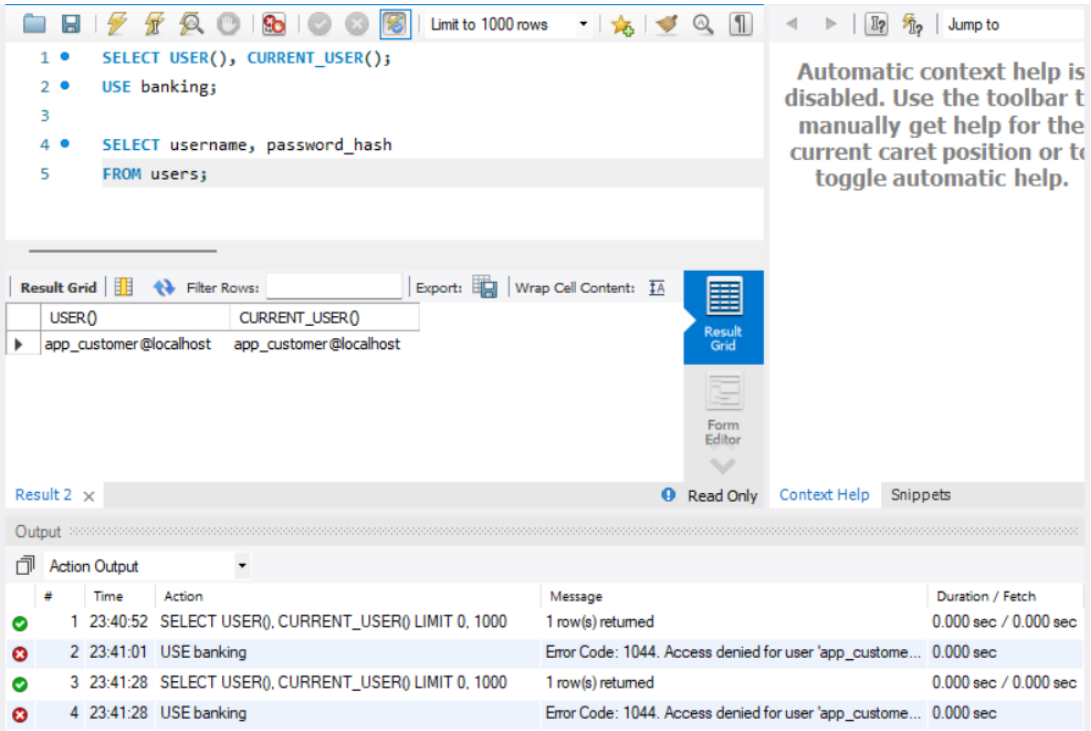


Figure 5.16: Customer-level database user cannot access the users table.

The app_customer database account attempted to read usernames and password hashes from

the users table. MySQL rejected the query with a SELECT command denied error because app_customer was not granted permission on the users table. This confirms that the least-privilege principle was correctly applied at the database layer.

5.6 CSRF Protection Test Results

To ensure that state-changing requests (POST) are protected against Cross-Site Request Forgery, the system's Double-Submit Cookie pattern was validated using both automated tools (Postman) and standard form submissions.

Table 5.7: CSRF Protection Test Cases

#	Action / Input	Expected Output	Result
5.6.1	Submit POST /login without _csrf token	HTTP 403 Forbidden, Request Denied	Pass
5.6.2	Submit POST /transfer with forged _csrf token	HTTP 403 Forbidden, CSRF Mismatch	Pass
5.6.3	Standard transfer submission via EJS form	Transaction processed successfully	Pass

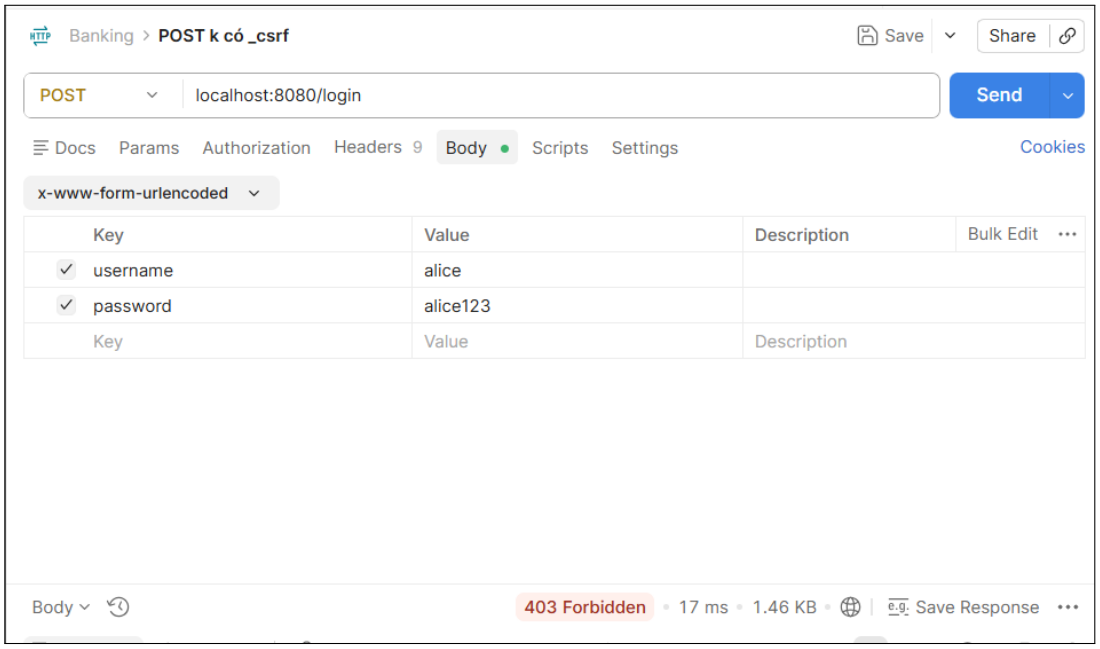


Figure 5.17: CSRF protection rejects POST /login without a valid _csrf token.

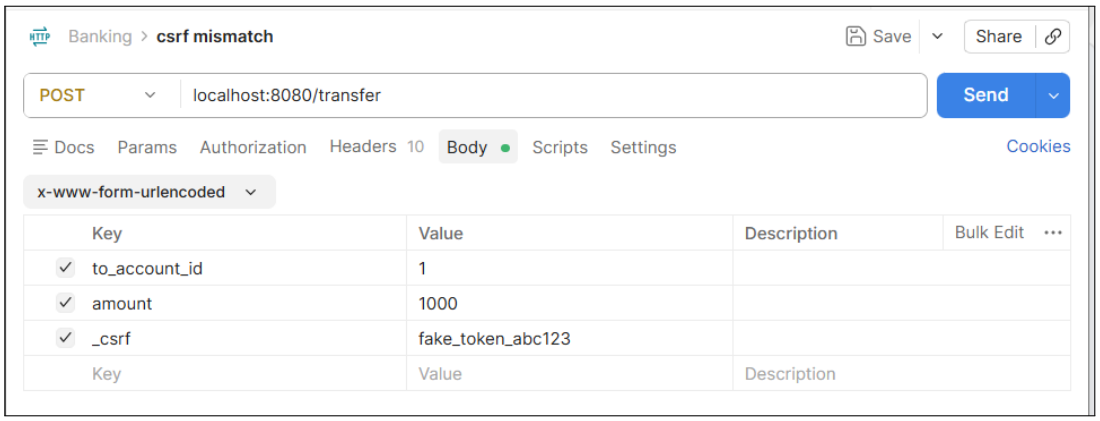


Figure 5.18: Forged CSRF token submitted in the POST /transfer request body.

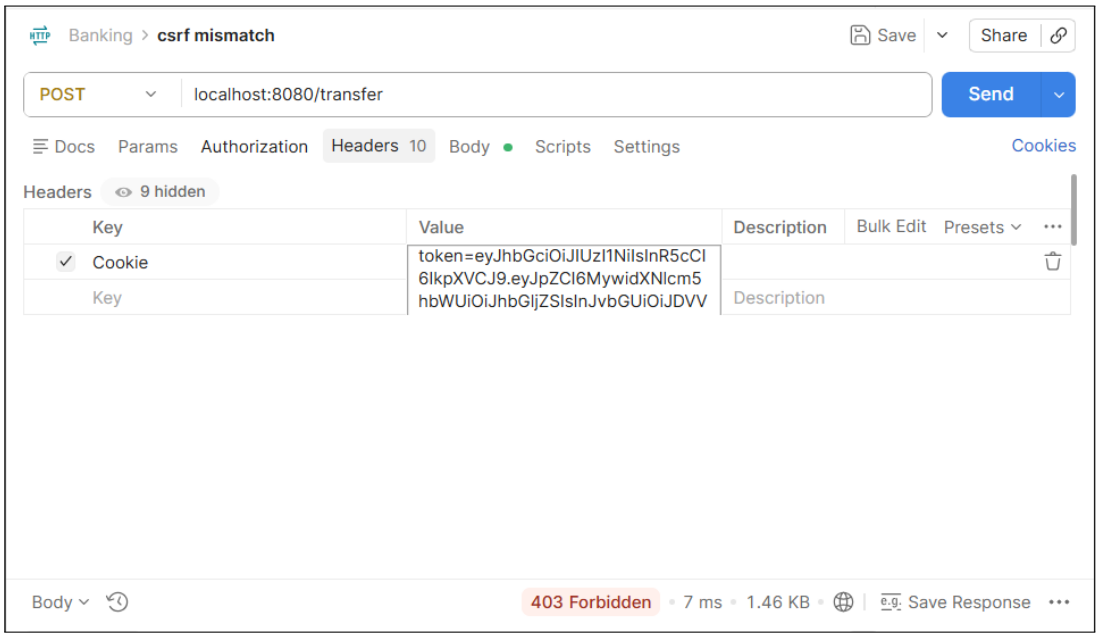


Figure 5.19: CSRF protection rejects the forged transfer request with a token mismatch error.

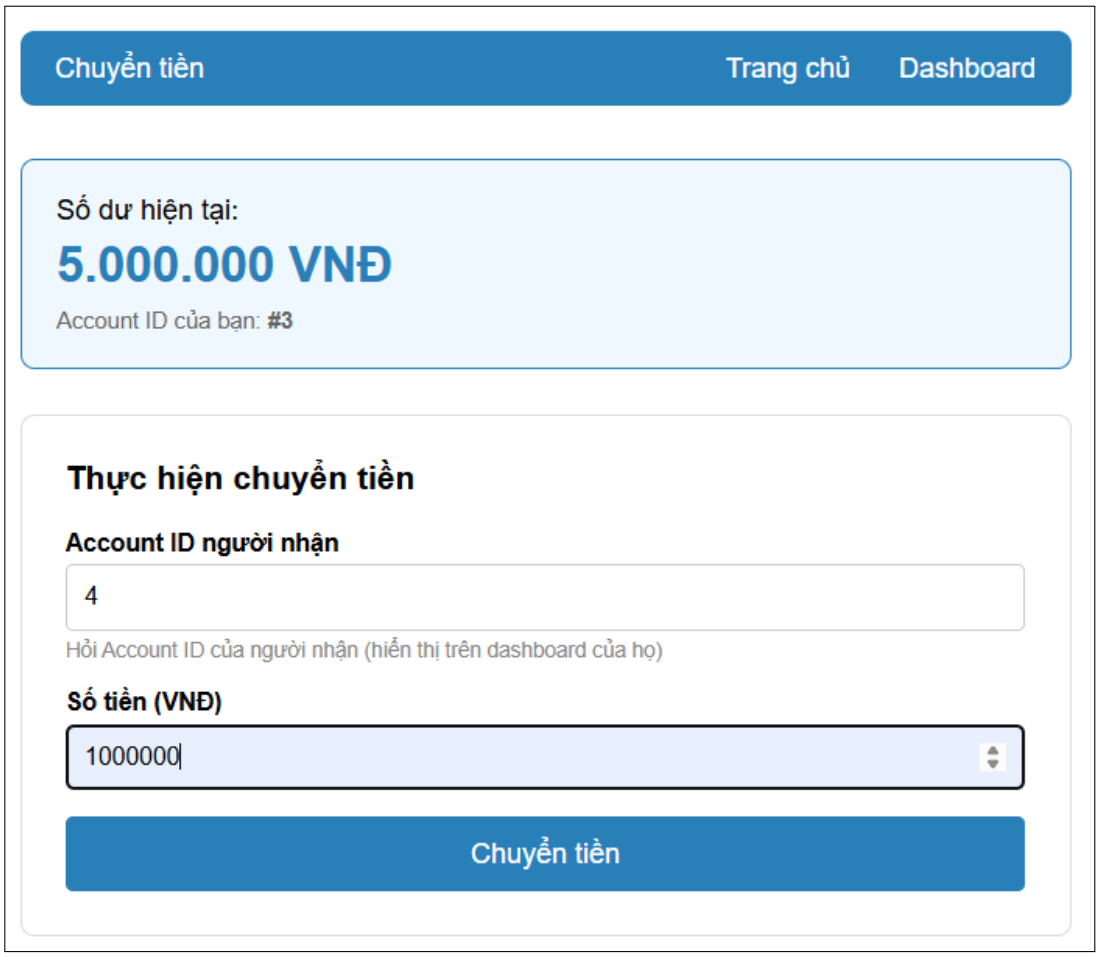


Figure 5.20: Standard transfer form includes a valid CSRF token before submission.

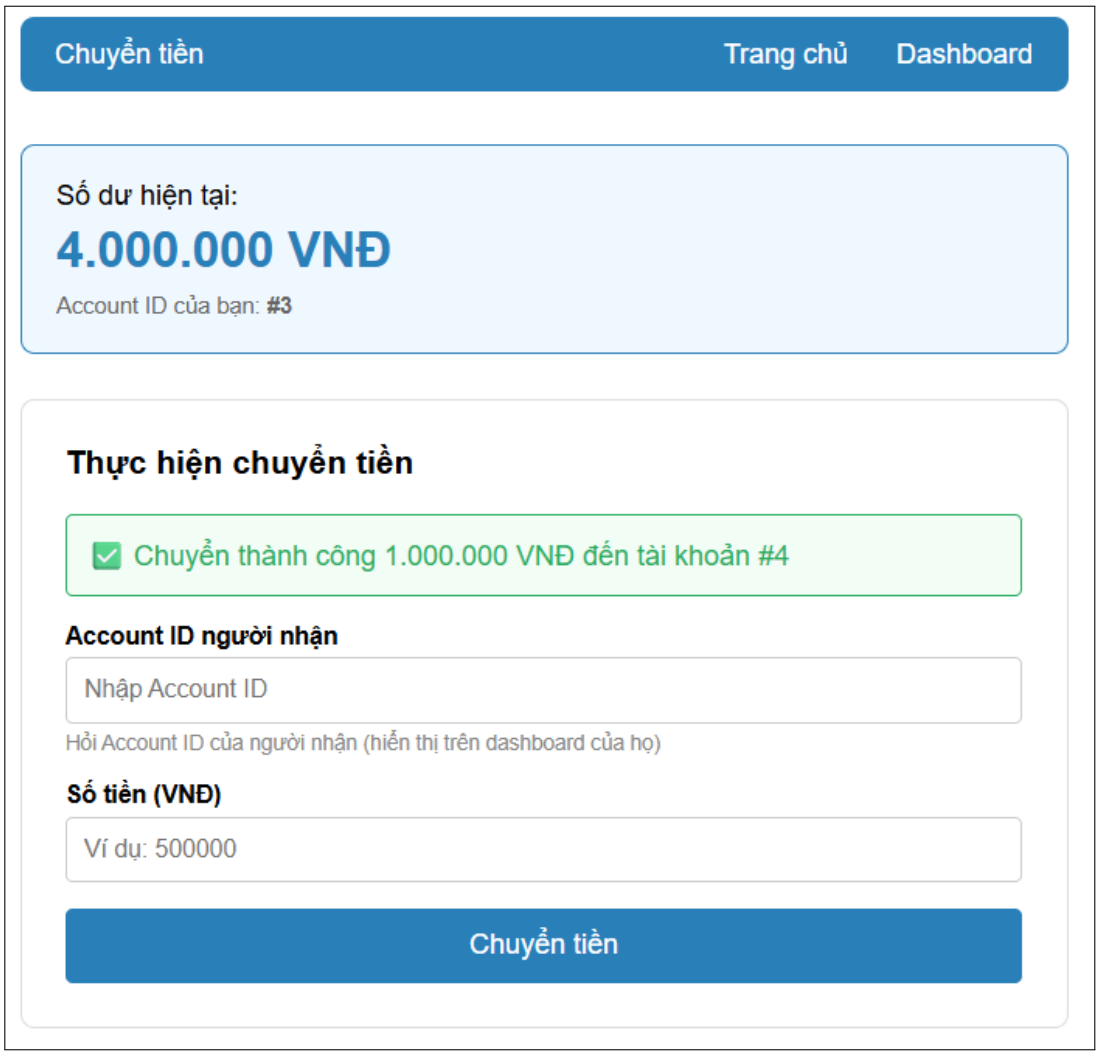


Figure 5.21: Standard transfer submission succeeds when the CSRF token is valid.

5.7 Transfer Validation and Concurrency Test Results

The fund transfer module was subjected to strict input validation, transaction limits, and race condition testing to prevent double-deduction vulnerabilities using database-level locking mechanisms.

Table 5.8: Transfer Validation and Concurrency Test Cases

#	Action / Input Scenario	Expected Output	Result
5.7.1	Transfer valid amount to another account	Balance deducted, TRANSFER logged	Pass
5.7.2	Transfer funds to own account ID	Error “Cannot transfer to yourself”	Pass
5.7.3	Transfer amount exceeding current balance	Error “Insufficient balance”	Pass
5.7.4	Transfer amount exceeding limit (>100M VND)	Error “Amount exceeds transaction limit”	Pass
5.7.5	Transfer to non-existent account ID	Error “Recipient account not found”	Pass
5.7.6	Concurrent requests (Race Condition Script)	Only 1 request succeeds (SELECT FOR UPDATE)	Pass

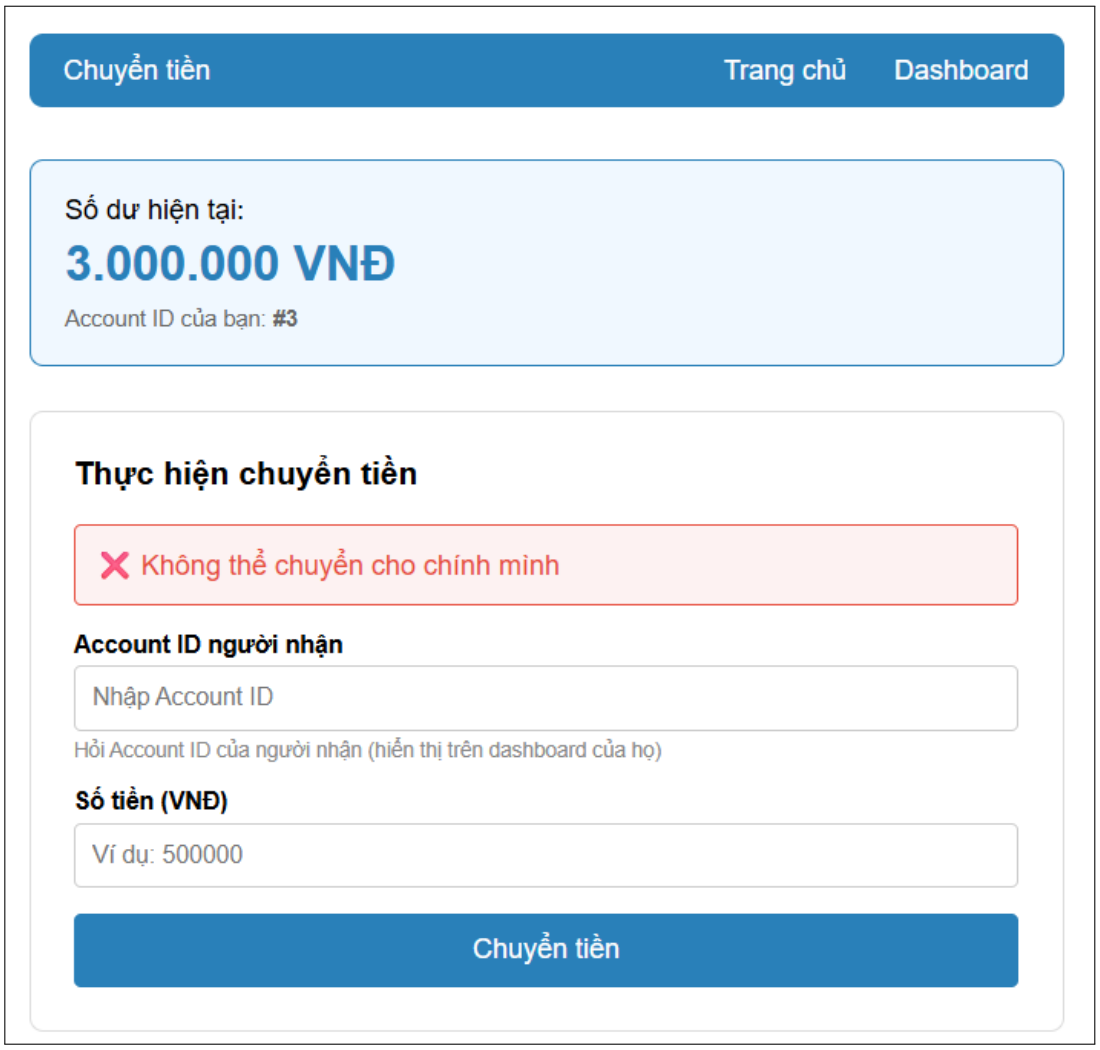


Figure 5.22: Transfer validation blocks a transfer to the same account.

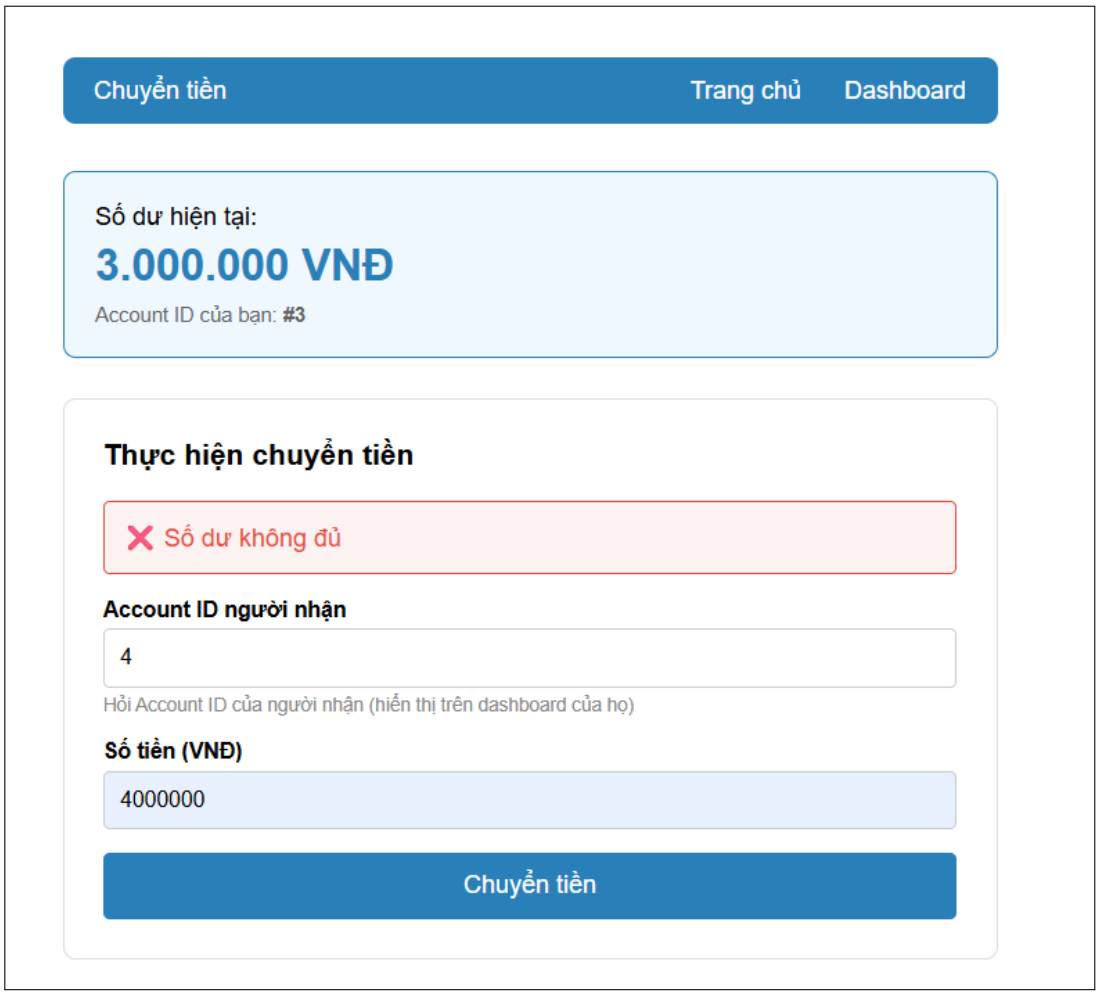


Figure 5.23: Transfer validation blocks a transfer amount exceeding the current balance.

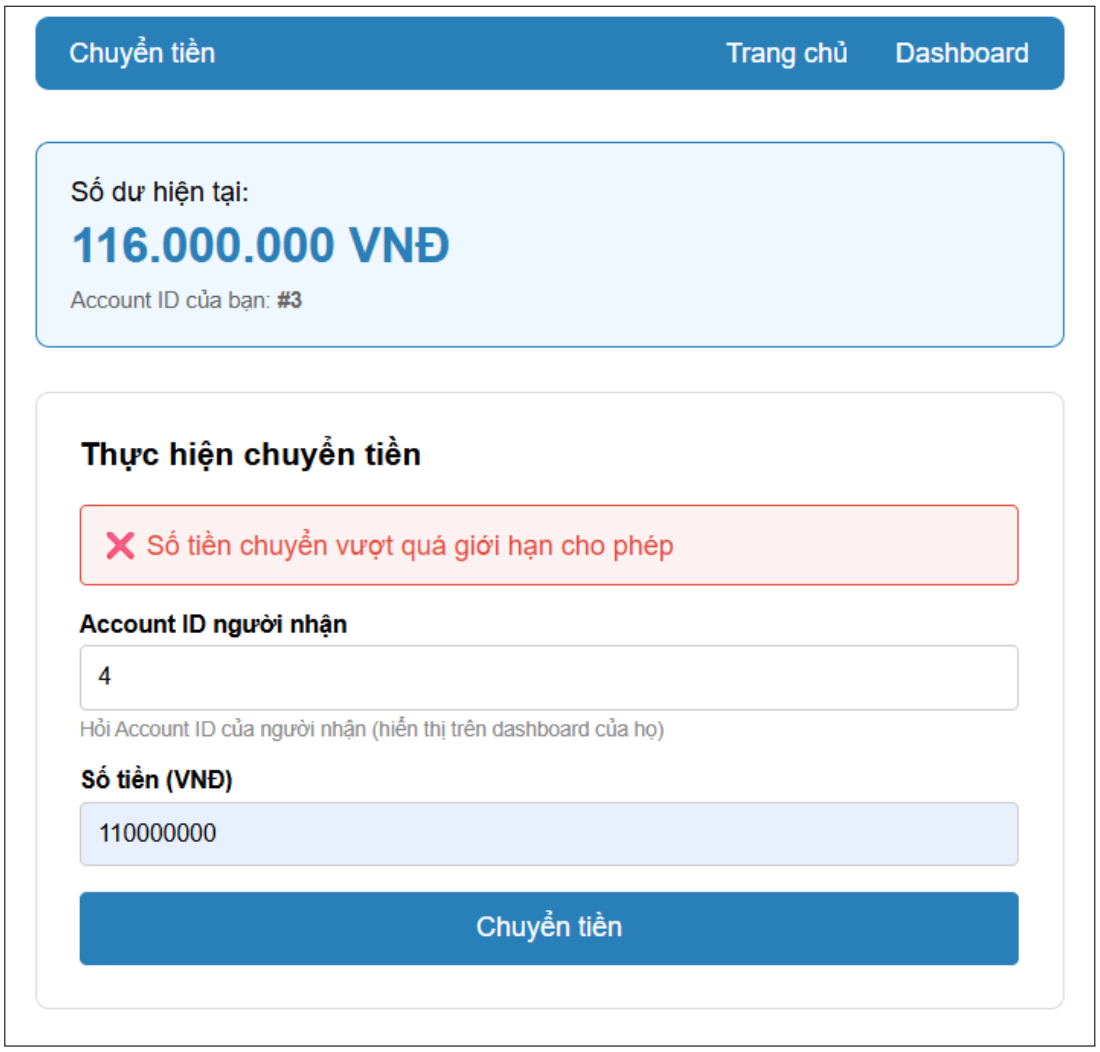


Figure 5.24: Transfer validation blocks a transfer amount exceeding the transaction limit.

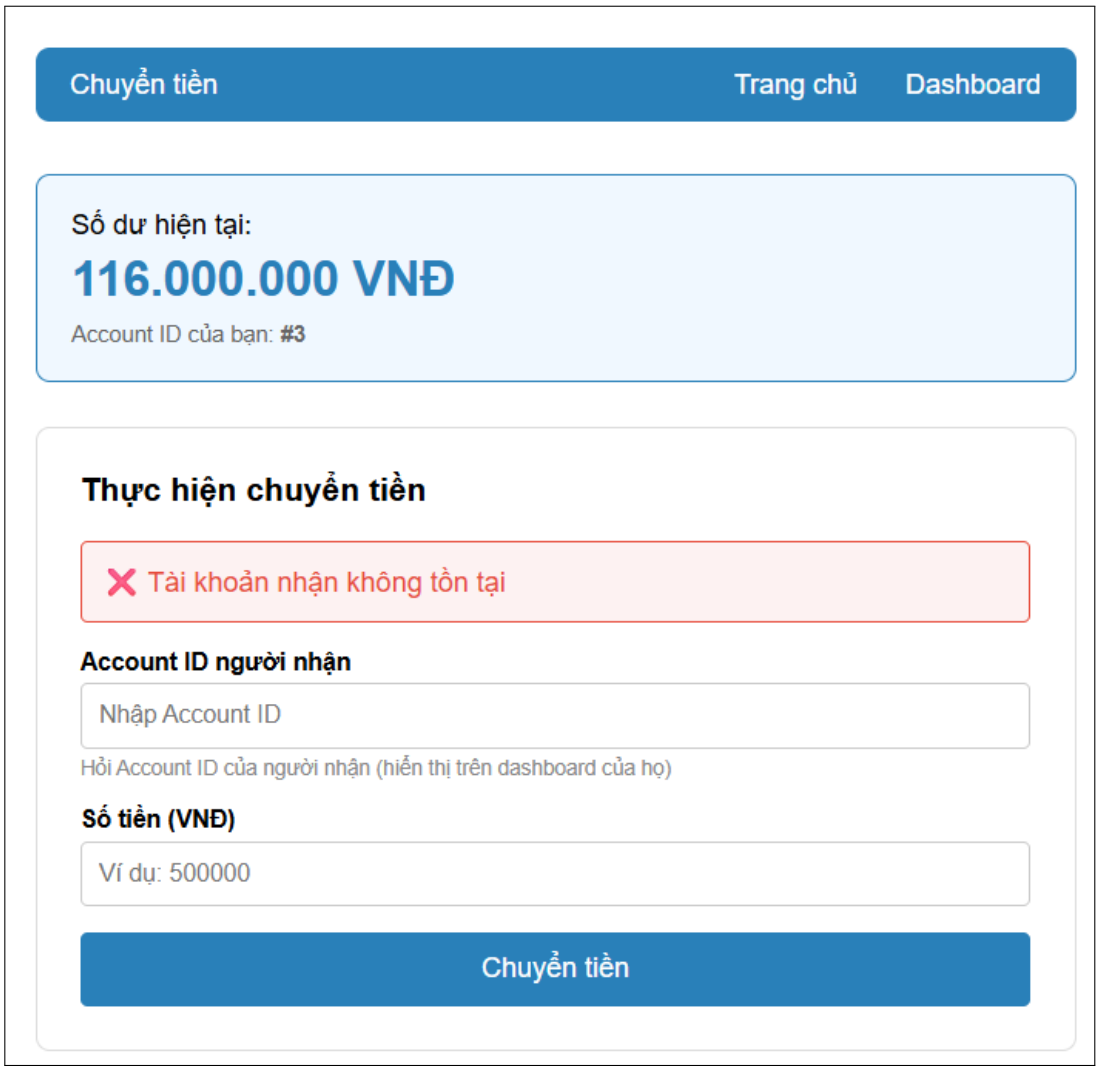
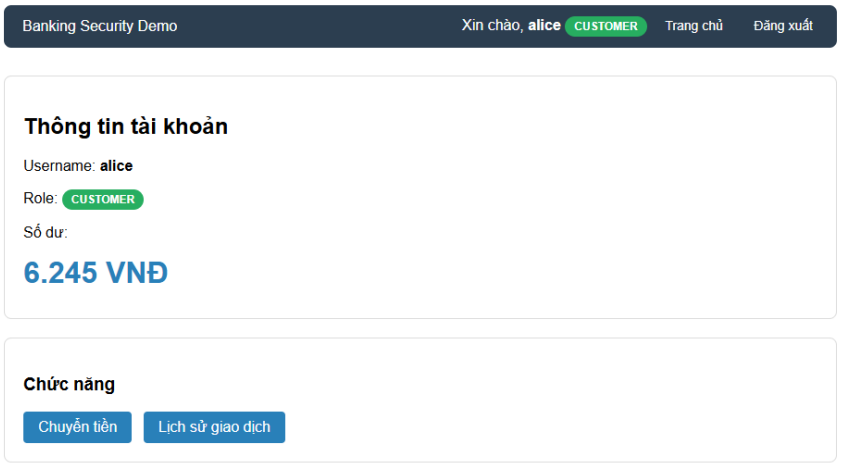


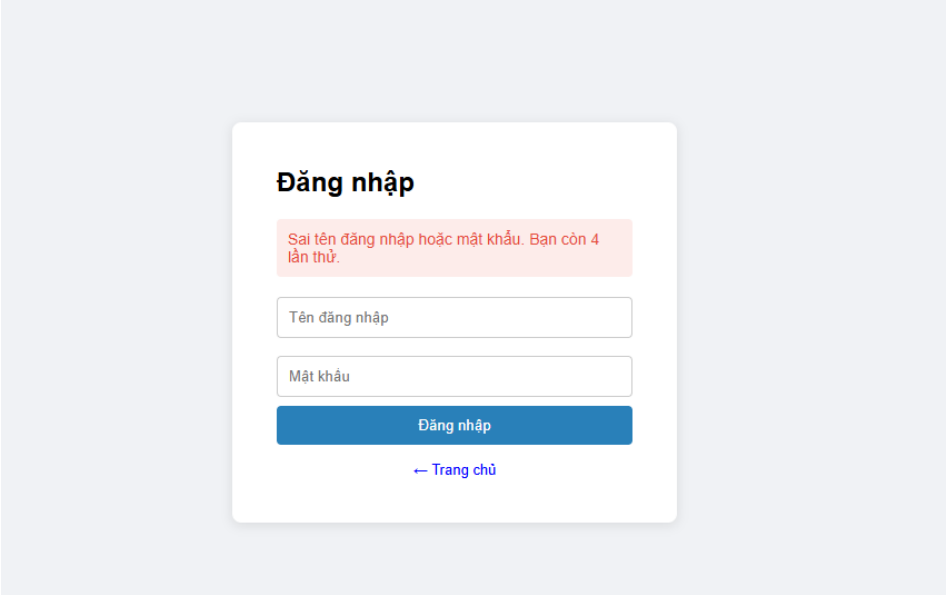
Figure 5.25: Transfer validation blocks a transfer to a non-existent recipient account.

5.8 Audit Log Verification

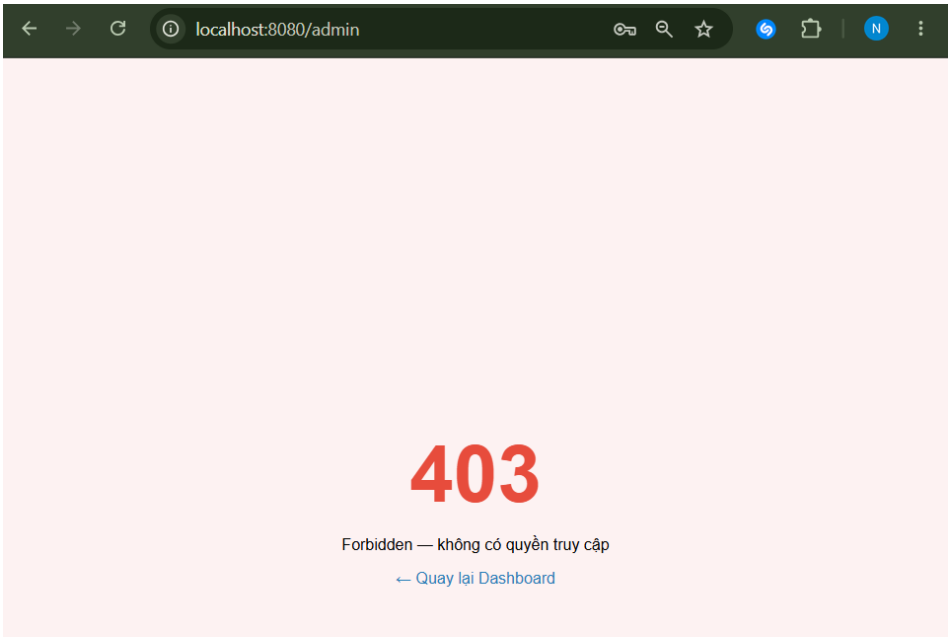
- Event 1 - LOGIN SUCCESS



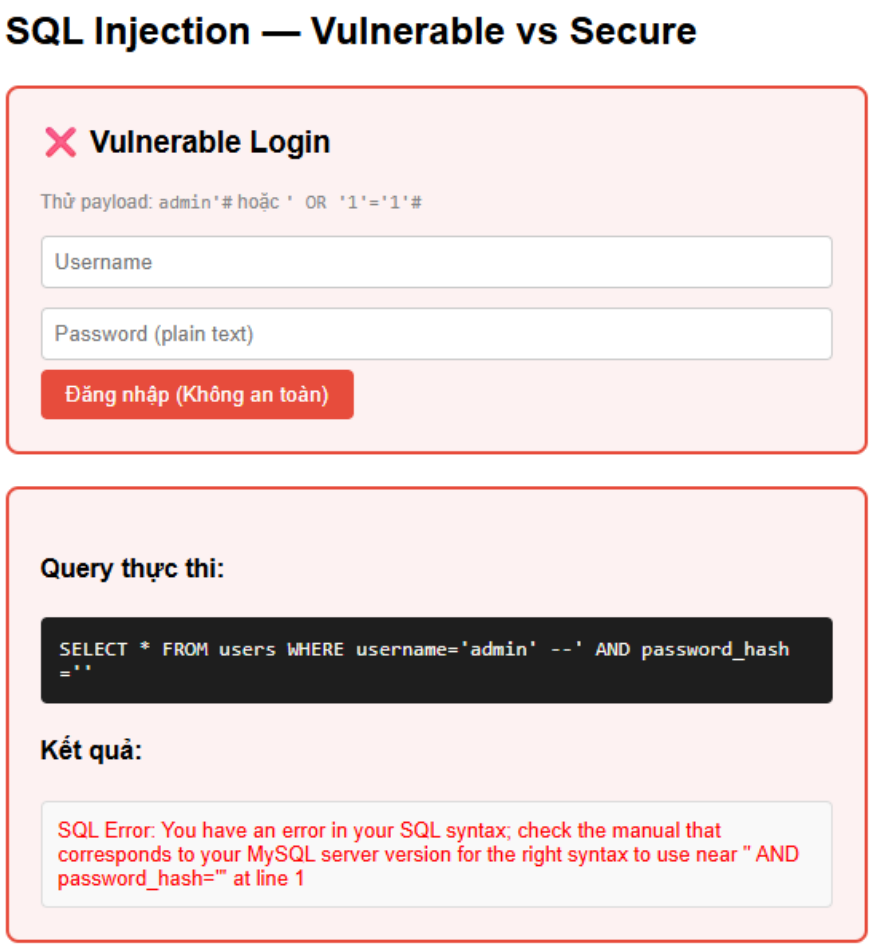
- Event 2 - LOGIN FAIL



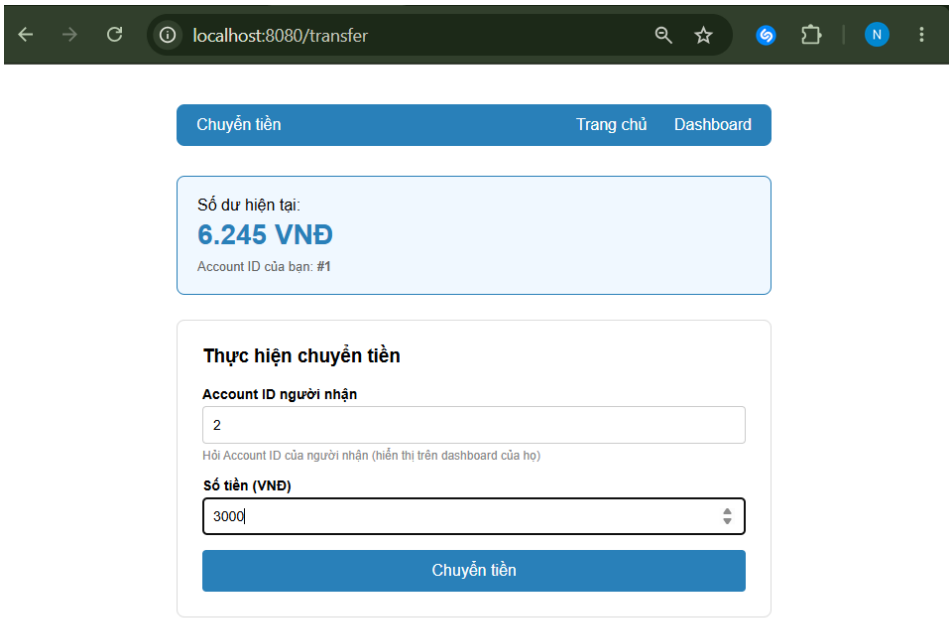
- Event 3 - UNAUTHORIZED ACCESS

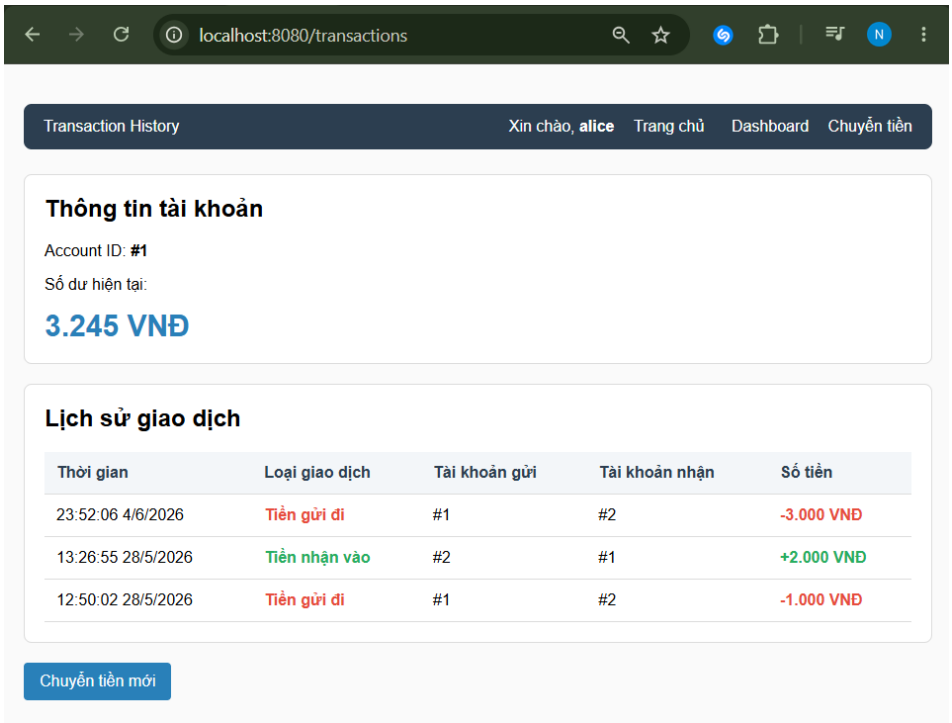


- Event 4 - SQL INJECTION ATTEMPT

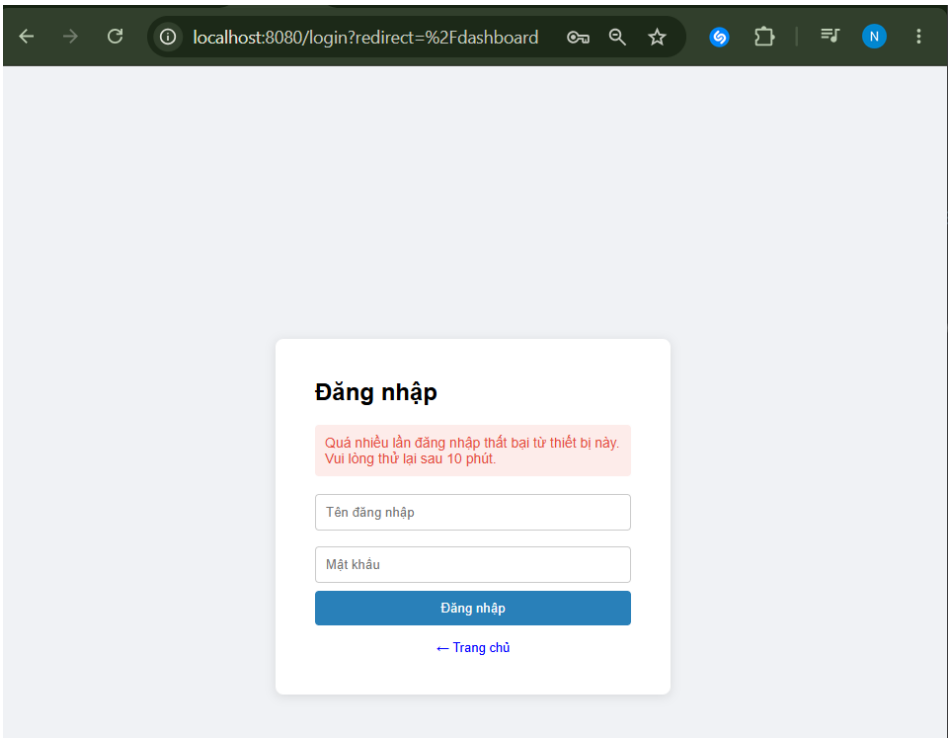


- Event 5 - TRANSFER





- Event 6 - ACCOUNT TEMPORARILY LOCKED



The audit log page records important security events, including successful login, failed login, unauthorized access, SQL Injection attempts, transfer actions, and account lockout events. Each entry includes the event type, username, IP address, details, and timestamp, which supports security monitoring and forensic investigation.

5.9 Summary of Test Results

Table 5.9: Overall Test Summary

Category	**Total Cases**	**Pass**	**Fail**
Role-Based Access Control (RBAC)	5	5	0
Account Lockout & Rate Limit	3	3	0
SQL Injection (Vulnerable Endpoints)	7	7	0
SQL Injection (Secure Endpoints)	4	4	0
Least-Privilege Database Access	3	3	0
CSRF Protection	3	3	0
Transfer Validation & Concurrency (Race Condition)	6	6	0
Total	**31**	**31**	**0**

The testing results show that the vulnerable module can be exploited through authentication bypass and UNION-based SQL Injection, while the secure module prevents the same payloads through ORM-based query handling and BCrypt verification. RBAC successfully restricts unauthorized role access, account lockout and rate limiting reduce brute-force risks, CSRF protection validates state-changing requests, transfer validation prevents invalid or concurrent

double-deduction scenarios, and least-privilege database users limit the potential impact of compromised credentials. Overall, the system demonstrates a defense-in-depth approach to database access control and SQL Injection prevention.

Chapter 6. Conclusion

6.1 Summary of Achievements

This project successfully implemented a Secure Online Banking Mini System as a demonstration platform for database access control and SQL Injection prevention. The system provides core banking functions such as user authentication, account dashboard, balance viewing, fund transfer, transaction history, administrative user management, and audit log monitoring. In addition to these functional modules, the project focused on implementing multiple security mechanisms to protect both the application layer and the database layer.

The main security achievements include Role-Based Access Control (RBAC), BCrypt password hashing, JWT-based authentication with secure cookie flags, account lockout after repeated failed login attempts, IP-based rate limiting, CSRF protection using the double-submit cookie pattern, secure HTTP headers with Helmet, audit logging, and MySQL least-privilege access control. The project also included an intentionally vulnerable SQL Injection module to demonstrate common attacks such as authentication bypass and UNION-based data extraction. These attacks were compared with secure implementations using Sequelize ORM and BCrypt verification.

The testing results show that the vulnerable module could be exploited by malicious SQL payloads, while the secure module successfully blocked the same attacks. RBAC prevented unauthorized users from accessing restricted pages, account lockout and rate limiting reduced brute-force risks, and audit logs recorded important security events. Overall, the project achieved its objective of demonstrating practical database security controls in a mini online banking environment.

6.2 Security Mechanisms Evaluation

The implemented security mechanisms were evaluated through manual testing using browser-based test cases and MySQL Workbench. The test results demonstrate that the system can prevent or reduce several common web and database security risks.

First, the SQL Injection prevention mechanism was effective. The vulnerable login and search forms showed that direct string concatenation can lead to authentication bypass and data leakage. However, the secure login and search forms prevented the same payloads by using Sequelize ORM and separating password verification from SQL query execution. This confirms that parameterized query handling is an effective defense against SQL Injection.

Second, RBAC correctly restricted access based on user roles. CUSTOMER users were denied access to administrative and audit pages, while AUDITOR users were allowed to view audit logs but were not allowed to perform money transfers. ADMIN users were able to access the user management page. These results confirm that the role-based middleware enforced access

control properly.

Third, authentication and password protection were strengthened through BCrypt hashing and JWT cookies. Passwords were never stored in plaintext, and authentication tokens were stored in cookies with security flags such as HttpOnly and SameSite. This reduced the risk of credential exposure and token theft.

Fourth, brute-force prevention was implemented at two levels. Account lockout protected individual accounts after repeated failed password attempts, while rate limiting reduced repeated login attempts from the same IP address. These two mechanisms complement each other because account lockout focuses on a specific account, while rate limiting focuses on suspicious request volume from a client.

Fifth, audit logging improved system visibility. Important events such as successful login, failed login, unauthorized access, SQL Injection attempt, money transfer, and account lockout were recorded with timestamp, username, IP address, and event details. This supports monitoring and forensic analysis.

Overall, the test results indicate that the implemented mechanisms worked as expected and demonstrate that the system follows a defense-in-depth approach rather than relying on only one security control.

6.3 Defense-in-Depth Analysis

The project applies defense-in-depth by combining multiple layers of protection. Each layer reduces risk at a different point in the attack path.

The first layer is Role-Based Access Control. RBAC prevents users from reaching functions outside their assigned role. For example, a CUSTOMER cannot access the Admin Panel or Audit Logs, and an AUDITOR cannot perform fund transfers. This layer reduces the chance that unauthorized users can interact with sensitive endpoints.

The second layer is least-privilege database access. MySQL users were configured with restricted permissions based on their purpose. For example, a customer-level database user is not allowed to read the users table containing password hashes. Therefore, even if a lower-privileged database account is compromised, the attacker cannot freely access all tables. This limits the damage of a successful compromise.

The third layer is secure query handling. Sequelize ORM and parameterized query handling prevent SQL Injection payloads from being interpreted as executable SQL code. Even when malicious input is submitted through the secure forms, the system treats it as normal input rather than part of the SQL statement. This prevents the injection from executing at all.

Additional layers further strengthen the system. BCrypt protects passwords even if hashes are exposed. JWT cookie flags reduce session-related risks. CSRF tokens prevent forged cross-site requests from performing sensitive actions. Account lockout and rate limiting reduce brute-force attacks. Audit logging provides traceability for suspicious activities. Together, these layers create a stronger security design than any single mechanism alone.

6.4 Limitations

Although the project successfully demonstrates important database and web application security mechanisms, it still has some limitations.

First, the system was developed and tested in a local demonstration environment. It was not deployed to a real production server, and HTTPS was not fully enforced during local testing. As a result, some production-level protections such as strict HSTS enforcement and real TLS certificate configuration were outside the current scope.

Second, the project focuses mainly on SQL Injection and relational database access control. Other attack categories such as NoSQL Injection, XML External Entity attacks, file upload attacks, and advanced API abuse were not included.

Third, security testing was performed manually using browser inputs, MySQL Workbench, and visible test results. Automated penetration testing tools and continuous security testing pipelines were not integrated.

Fourth, the SQL Injection vulnerable module was intentionally included for demonstration purposes. Although it is protected by demo mode, it should not be enabled in a real production environment.

Fifth, CSRF protection and rate limiting were tested in a local environment. In a real distributed deployment, rate limiting should use a shared storage mechanism such as Redis instead of in-memory counters to ensure consistent behavior across multiple server instances.

Finally, the system is a mini banking prototype. It does not include all features required by a real banking application, such as two-factor authentication, transaction approval **workflow**, fraud detection, encryption at rest, real payment gateway integration, and regulatory compliance controls.

6.5 Future Work

Several improvements can be added in future development to make the system more secure and closer to a production-ready banking application.

First, the system should be deployed with HTTPS and strict TLS configuration. Secure cookies should always use the Secure flag in production, and HTTP Strict Transport Security should be enabled after HTTPS deployment.

Second, two-factor authentication should be added to strengthen the login process. This would reduce the risk of account compromise even if a password is guessed or leaked.

Third, automated security testing should be integrated. Tools for static code analysis, dependency vulnerability scanning, and dynamic web application testing could help detect security issues earlier.

Fourth, rate limiting should be improved with a centralized store such as Redis. This would allow the protection to work correctly in a multi-server deployment.

Fifth, the database access model can be improved further by separating runtime accounts from

migration or administration accounts. The application should not use a root or high-privilege account during normal operation.

Sixth, the audit logging module can be extended with filtering, exporting, severity levels, and alert notifications. For example, repeated SQL Injection attempts or account lockout events could trigger an administrator alert.

Seventh, the banking features can be expanded with transfer approval, transaction limits, account statements, email notifications, and fraud detection rules. These additions would make the system more realistic while still maintaining the focus on security.

In conclusion, this project achieved its main goal of demonstrating database access control and SQL Injection prevention in a mini online banking system. The implemented security controls show how layered defenses can reduce the risk of common attacks and provide a foundation for future secure system development.

Bibliography

[1] National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). https://csrc.nist.gov/pubs/sp/800/218/final

[2] OWASP Foundation. (2021). *A03:2021 - Injection*. *OWASP Top 10*. https://owasp.org/Top10/A03_2021-Injection/

[3] OWASP Foundation. (n.d.). *SQL injection*. https://owasp.org/www-community/attacks/SQL_Injection

[4] OWASP Foundation. (n.d.). *SQL injection prevention cheat sheet*. *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[5] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer, 29*(2), 38-47.

[6] Oracle Corporation. (n.d.). *Access control and account management*. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/access-control.html

[7] Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*.

[8] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON web token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

[9] OWASP Foundation. (n.d.). *Authentication cheat sheet*. *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[10] OWASP Foundation. (n.d.). *Cross-site request forgery (CSRF)*. https://owasp.org/www-community/attacks/csrf

Chapter A. Full Database Schema

Listing A.1: Complete Database Schema

Listing A.1: Complete Database Schema

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('CUSTOMER', 'ADMIN', 'AUDITOR') NOT NULL,
    failed_login_attempts INT NOT NULL DEFAULT 0,
    locked_until DATETIME NULL DEFAULT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    createdAt DATETIME DEFAULT NOW(),
    updatedAt DATETIME DEFAULT NOW()
);

CREATE TABLE accounts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    balance DECIMAL(15, 2) DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE transactions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    from_account INT,
    to_account INT,
    amount DECIMAL(15, 2),
    createdAt DATETIME DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    event_type VARCHAR(50),
    username VARCHAR(100),
    ip_address VARCHAR(50),
    details TEXT,
    createdAt DATETIME DEFAULT NOW()
);
```